

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Le Xuan An

Gradient-Routed Alignment with Capacity-aware Experts
for Multimodal Understanding

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



LE XUAN AN

**Gradient-Routed Alignment with Capacity-aware Experts
for Multimodal Understanding**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Giảng viên hướng dẫn: Tiến sĩ Nguyễn Văn Sơn

Giảng viên đồng hướng dẫn: Giáo sư Võ Đình Hiếu

HÀ NỘI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Le Xuan An

**Gradient-Routed Alignment with Capacity-aware Experts
for Multimodal Understanding**

BACHELOR'S THESIS

Major: Computer Science

Advisor: Dr. Nguyen Van Son

Co-Supervisor: Professor Vo Dinh Hieu

HANOI - 2026

Lời cảm ơn

Assurance

I hereby affirm that this thesis is my own original work. I have not submitted it previously for any academic or professional award.

I completed this thesis through my own efforts, with important guidance and support from Dr. Nguyen Van Son and Professor Vo Dinh Hieu for graduation purposes. I confirm that all information, data and findings presented here are original and free from plagiarism.

I have acknowledged every source that I consulted and included each one in the references. I accept full responsibility for the accuracy and integrity of this thesis under the regulations of VNU University of Engineering and Technology.

Hanoi, Ngày 22 tháng 4 năm 2026

Student

Le Xuan An

Tóm tắt

Khóa luận này nghiên cứu chứng cất tri thức đa giáo viên không đồng nhất cho các mô hình thị giác-ngôn ngữ (VLM) nhỏ gọn. Bài toán khó vì các teacher đến từ nhiều họ mô hình khác nhau, dùng tokenizer khác nhau, và không teacher nào hữu ích như nhau trên mọi mẫu. Vì vậy, cách lấy trung bình cố định giữa các teacher thường kém hiệu quả, đặc biệt với các student dưới một tỷ tham số.

Khóa luận đề xuất Gradient-Routed Alignment with Capacity-aware Experts (GRACE). Khung này xử lý trực tiếp ba vấn đề thực tế: lệch tokenizer, khác biệt thể mạnh giữa các teacher và bất ổn sau bước routing. Phương pháp kết hợp hàm mất mát trie-Wasserstein trên cây byte dùng chung, teacher gate phụ thuộc vào student, sparse top- k routing có ràng buộc capacity và cơ chế tinh chỉnh trọng số GRACE dựa trên gradient. Gate trước hết chọn một tập teacher nhỏ cho từng mẫu. Sau đó, GRACE so sánh gradient KD của từng teacher đã được chọn với gradient CE có giám sát của student, làm mượt điểm số bằng exponential moving average rồi chuyển các điểm này thành trọng số KD cuối cùng.

Thực nghiệm trên TextVQA, DocVQA và ChartQA cho thấy GRACE luôn tốt hơn finetuning trực tiếp và các baseline chứng cất đã so sánh. Trong thiết lập bốn teacher được báo cáo, GRACE đạt 69.38, 73.11 và 80.20 với SmolVLM-500M, và 60.73, 60.48 và 72.32 với SmolVLM-256M. Với SmolVLM-500M, mức cải thiện tương ứng so với mô hình gốc là 22.26%, 22.83% và 27.71%. Phương pháp cũng thu hẹp 55.03% khoảng cách tới teacher trên TextVQA, 39.53% trên DocVQA và 55.34% trên ChartQA.

Từ khóa: mô hình thị giác-ngôn ngữ, chứng cất tri thức, đa giáo viên, mixture-of-experts routing, chứng cất Wasserstein khác tokenizer

Abstract

We study heterogeneous multi-teacher distillation for compact visual language models (VLMs). The setting is difficult because the teachers come from different model families, use different tokenizers and help different samples in different ways. Fixed teacher averaging fails, especially for sub-billion students.

We present GRACE, short for Gradient-Routed Alignment with Capacity-aware Experts. The framework directly addresses three practical failures at once: tokenizer mismatch, heterogeneous teacher specialization and unstable post-routing weighting. To the best of our knowledge, it is the first distillation framework to employ a shared tree-metric OT for heterogeneous multi-teacher VLM distillation. It combines a trie-Wasserstein KD loss on a shared byte trie, a student-conditioned teacher gate, sparse top- k capacity-aware routing and GRACE score refinement. The gate first selects a small teacher subset for each sample. GRACE then compares each routed teacher’s KD gradient with the student’s supervised CE gradient, smooths the agreement scores with an exponential moving average and converts them into final KD weights.

Experiments on TextVQA, DocVQA and ChartQA demonstrate that GRACE consistently improves over direct finetuning and the compared distillation baselines. In the reported four-teacher setting, GRACE reaches 69.38, 73.11 and 80.20 on SmolVLM-500M and 60.73, 60.48 and 72.32 on SmolVLM-256M. For SmolVLM-500M, this corresponds to gains of 22.26%, 22.83% and 27.71% over the original student. The method also closes 55.03% of the original teacher gap on TextVQA, 39.53% on DocVQA and 55.34% on ChartQA. These results establish GRACE as a strong distillation framework for compact text-rich VLMs.

Keywords: visual language model, knowledge distillation, multi-teacher distillation, mixture-of-experts routing, cross-tokenizer Wasserstein distillation

Table of Contents

Lời cảm ơn

Assurance i

Tóm tắt ii

Abstract iii

Table of Contents iv

List of Figures vii

List of Tables ix

List of Abbreviations x

Chapter 1 Introduction 1

1.1 Background and Motivation 1

1.2 Related Work 6

1.2.1 Compact VLM Distillation with Matched Output Spaces 7

1.2.2 Cross-Tokenizer and Transport-Based Distillation 7

1.2.3 Representation Matching Beyond Logits 7

1.2.4 Multi-Teacher Distillation and Adaptive Teacher Selection 8

1.2.5 Research Gap 8

1.3 Problem Statement and Research Objectives 8

1.4 Research Questions 9

1.5 Thesis Contributions 10

1.6 Thesis Organization 10

Chapter 2 Preliminaries 11

2.1 Transformer-Based Vision-Language Models 11

2.1.1 Transformer Basics 11

2.1.2	Vision Encoder and Multimodal Fusion	11
2.2	Knowledge Distillation	12
2.2.1	Teacher-Student Learning	12
2.2.2	Temperature-Scaled Output Distributions	13
2.3	Cross-Tokenizer Distillation Background	13
2.3.1	ULD as a Prior Baseline	13
2.3.2	Optimal Transport and Tree-Structured Mass Matching	14
2.3.3	Answer-Token Distillation	16
2.4	Multi-Teacher Distillation	16
2.4.1	Per-Teacher KD Loss	16
2.4.2	Learned Routing and Load Balancing	16
Chapter 3	Proposed Method	19
3.1	Method Overview and Problem Formulation	19
3.1.1	Notation Summary	20
3.1.2	Training Step	21
3.2	Trie-Wasserstein Cross-Tokenizer Distillation	21
3.2.1	Supervised Answer Tokens	21
3.2.2	Shared Byte Trie	23
3.2.3	Sparse Top- k Support and Tail Edge	24
3.3	Learned Teacher Gate and Load Balancing	26
3.3.1	Student-Conditioned Router	26
3.3.2	Top- k , Capacity and Fallback	27
3.3.3	Balancing Loss and Expert Bias Update	28
3.4	GRACE Score Refinement	30
3.4.1	Gradient Proxies	30
3.4.2	GRACE Scores and Logged Active Flags	30
3.4.3	Warmup, Score Smoothing and Gradient Weights	31
3.4.4	Router-Gradient Blend and Uniform Fallback	32
3.4.5	Sample-Wise KD Aggregation	35
3.5	Final Objective	35
Chapter 4	Evaluation Methodology	36
4.1	Experimental Setup	36
4.2	Datasets	36
4.2.1	Dataset Descriptions	37
4.2.2	Dataset Size and Splits	37

4.3	Training Configuration	39
4.4	Evaluation Metrics	41
4.5	Baselines and Compared Methods	42
4.6	Implementation Details	45
4.6.1	Offline Teacher-Logit Preparation	45
4.6.2	Remote Logit Streaming During Training	46
Chapter 5	Experiment Results	48
5.1	Main Comparison Results	48
5.1.1	Baseline Results	48
5.1.2	Overall Comparison	48
5.2	Representation Similarity Analysis	60
5.3	Ablation Study	62
5.4	Efficiency Analysis	63
5.5	Discussion	65
5.6	Limitations and Threats to Validity	65
	Conclusion and Future Work	67
	References	68

List of Figures

1.1	CLIP-style vision-language pretraining with a shared image-text embedding space [1, 2].	2
1.2	Scaling-law trends from Kaplan et al.	2
1.3	SmolVLM memory comparison from the Hugging Face release.	3
1.4	Representative compact VLM scales.	4
1.5	TinyLLaVA vs. LLaVA-1.5-7B benchmark comparison.	4
1.6	Best proprietary vs. open-source frontier results on three benchmarks.	4
1.7	Low-bit quantization can sharply reduce VLM accuracy.	5
1.8	Teacher-student knowledge distillation.	6
2.1	Transformer and ViT building blocks.	12
2.2	Representative architecture families of modern vision-language models.	13
2.3	Continuous displacement view of optimal transport [3, 4].	14
2.4	Tree-structured and matrix views of discrete optimal transport.	15
2.5	Sparse routing with expert capacity constraints.	18
2.6	Sparse routing and expert specialization in MoE.	18
3.1	Architecture view of GRACE.	23
3.2	Shared-trie view of cross-tokenizer matching for <code>document/receipt</code> and <code>doc/rece</code>	24
3.3	Sparse trie-Wasserstein support in the implementation.	25
3.4	Trie split for <code>document</code> and <code>doc</code>	25
3.5	Student-conditioned router sketch in GRACE.	27
3.6	Bias-feedback behavior in GRACE.	29
3.7	Routing dynamics in GRACE.	29
3.8	Directional view of gradient descent.	31
3.9	EMA smoothing in GRACE.	31
3.10	Warmup schedule in GRACE.	32
3.11	GRACE score refinement in GRACE.	33
3.12	GRACE weighting cases.	33
3.13	Weight refinement in GRACE.	34

3.14	Sample-specific weighting in GRACE.	34
4.1	TextVQA sample image.	37
4.2	DocVQA sample image.	38
4.3	ChartQA sample image.	38
4.4	R2 bucket used for teacher-logit storage.	46
4.5	Training with remote teacher logits.	47
5.1	Grouped comparison of original, best-baseline and GRACE results.	54
5.2	DEX-AR overview.	55
5.3	Representative TextVQA attention example.	56
5.4	Training-curve comparison for uniform weighting and GRACE.	57
5.5	Representative collapse and non-collapse routing examples.	58
5.6	Final-layer CKA heatmaps across two benchmarks.	61
5.7	Vision-layer CKA heatmaps across two benchmarks.	61

List of Tables

3.1	Main symbols used in Chapter 3.	21
3.2	Illustrative ULD and trie-Wasserstein comparison.	26
4.1	Dataset split sizes for the three benchmarks.	39
4.2	Main training protocol for the active distillation launch path.	40
4.3	Baseline groups used in the evaluation chapter.	42
4.4	Engineering settings for the active logit-streaming pipeline.	47
5.1	Original SmolVLM student results.	49
5.2	Teacher and reference-model results.	49
5.3	TextVQA main comparison.	51
5.4	DocVQA main comparison.	52
5.5	ChartQA main comparison.	53
5.6	Relative gains of GRACE on SmolVLM-500M.	59
5.7	Gap-to-teacher analysis for SmolVLM-500M.	59
5.8	DocVQA component ablation.	63

List of Abbreviations

Abbr.	Full Term	Description
ANLS	Average Normalized Levenshtein Similarity	Evaluation metric used for DocVQA.
API	Application Programming Interface	Interface used to query model outputs in black-box settings.
bf16	Brain Floating Point 16	Low-precision floating-point format used for efficient mixed-precision training.
BLIP-2	Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models	A modular vision-language model with a Q-Former bridge.
CCA	Canonical Correlation Analysis	Classical similarity analysis method related to the interpretation of CKA scores.
CE	Cross-Entropy	Supervised loss used for classification or distillation objectives.
CKA	Centered Kernel Alignment	Representation similarity measure used to compare hidden layers between teacher and student models.
ChartQA	Chart Question Answering	Benchmark for question answering over charts.
CLIP	Contrastive Language-Image Pre-training	Dual-encoder vision-language pretraining framework.
CNN	Convolutional Neural Network	Standard visual backbone architecture used as a comparison point to ViT.
DocVQA	Document Visual Question Answering	Benchmark for question answering on document images.
EOS	End of Sequence	Special token that marks the end of a generated answer span.

Abbr.	Full Term	Description
FFN	Feedforward Network	Transformer sublayer applied after attention.
fp16	Floating Point 16	Half-precision floating-point format used in mixed-precision training or inference.
GRACE	Gradient-Routed Alignment with Capacity-aware Experts	Proposed multi-teacher distillation method in this thesis.
GPU	Graphics Processing Unit	Parallel hardware accelerator used for training and serving deep learning models.
SHM-CKA	Hidden-State Matching with CKA	Baseline distillation method that aligns teacher and student hidden representations using CKA.
InfoNCE	Information Noise-Contrastive Estimation	Contrastive objective for aligning image and text representations.
JSON	JavaScript Object Notation	Lightweight structured data format used for the LLaVA-style training files in this thesis.
JS	Jensen-Shannon	Symmetric divergence measure between probability distributions.
KD	Knowledge Distillation	Transfer of knowledge from a teacher model to a student model.
KL	Kullback-Leibler	Measure used to compare probability distributions.
KLD	Kullback-Leibler Divergence	Divergence-based objective widely used in distillation.
LLaVA	Large Language and Vision Assistant	Instruction-tuned projector-based vision-language model.
LLM	Large Language Model	Language model with large parameter scale and strong reasoning ability.
LN	Layer Normalization	Normalization method used to stabilize Transformer training.
MLP	Multi-Layer Perceptron	Feedforward projection block used in Transformers and classifiers.

Abbr.	Full Term	Description
MMD	Maximum Mean Discrepancy	Similarity metric used for feature-level alignment.
MoE	Mixture of Experts	Sparse architecture that routes tokens to selected experts.
MSA	Multi-Head Self-Attention	Attention block that models token interactions in Transformers.
MTKD	Multi-Teacher Knowledge Distillation	Distillation setting that uses more than one teacher model.
NLP	Natural Language Processing	Field concerned with computational understanding of language.
NLS	Normalized Levenshtein Similarity	String similarity score used in DocVQA evaluation.
OCR	Optical Character Recognition	Extraction of text from images or scanned documents.
OT	Optimal Transport	Distribution-matching perspective used to motivate tokenizer-agnostic distillation.
TWD	Tree-Wasserstein Distance	Tree-structured optimal-transport distance used as background for the trie-Wasserstein loss in this thesis.
PE	Positional Encoding	Representation of token order in Transformer models.
PLN	Parametric Layer Normalization	Layer normalization variant with trainable scale and bias.
Q-Former	Querying Transformer	Bridging module used by BLIP-2 to connect vision and language features.
QA	Question Answering	Task of generating answers to natural language questions.
ReLU	Rectified Linear Unit	Common non-linear activation function.
RMSNorm	Root Mean Square Layer Normalization	Lightweight normalization method based on root mean square statistics.
RoPE	Rotary Positional Embedding	Positional encoding method based on rotational transformations.

Abbr.	Full Term	Description
TF32	TensorFloat-32	NVIDIA tensor-core numeric format used to accelerate matrix operations during training.
TextVQA	Text Visual Question Answering	Benchmark for question answering on natural images containing text.
ULD	Universal Logit Distillation	Tokenizer-agnostic distillation loss based on ranked probability distributions.
ViT	Vision Transformer	Transformer-based model for image representation learning.
VLM	Vision-Language Model	Model that jointly processes visual and textual inputs.
VLM-Eval-Kit	Vision-Language Model Evaluation Kit	Toolkit used in this thesis to run standardized multimodal evaluation.
VNU	Vietnam National University	Parent university system of VNU University of Engineering and Technology.
VQA	Visual Question Answering	Task of answering questions grounded in visual inputs.
ZeRO	Zero Redundancy Optimizer	Memory-optimization technique used by DeepSpeed to reduce optimizer-state replication during large-model training.

Chapter 1

Introduction

This chapter introduces the problem, motivates the need for compact vision-language model distillation and states the main research objectives. It first explains why deployment constraints make sub-billion students important, then positions the work against prior methods. It ends by defining the technical gap that motivates the proposed approach.

1.1 Background and Motivation

The core task here is to distill strong multimodal ability from large VLM teachers into **sub-billion** VLM students. The goal is to do so without losing performance on text-centric benchmarks. Vision-Language Models (VLMs) jointly process images and text for tasks such as image captioning, visual question answering, document understanding and instruction following [2, 5–8].

Multimodal understanding here refers to practical capabilities such as reading text in images, understanding the input question and performing basic reasoning over visual and textual evidence, not broad general intelligence [9–11]. Our focus is efficient transfer of this ability to compact students.

Many modern VLM families still build on the basic vision-text pretraining pattern shown in Figure 1.1.

However, high-performing VLMs are often large and expensive in memory, latency and deployment cost [5, 6, 12, 13]. This cost limits their use in resource-constrained settings.

This pressure is consistent with scaling-law results. Kaplan et al. show that model quality improves predictably with scale, while Hoffmann et al. show that compute-optimal training still requires jointly increasing model size and data tokens [14, 15]. Larger models therefore remain attractive for accuracy, but they do not become cheap to train or deploy.

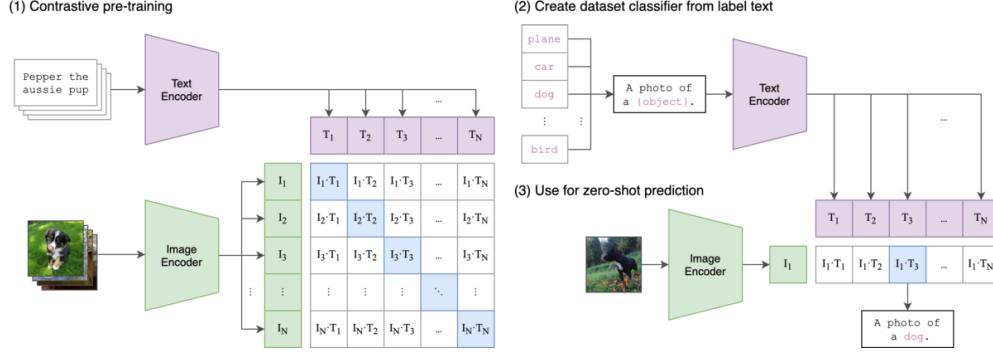


Figure 1.1: CLIP-style vision-language pretraining with a shared image-text embedding space [1, 2].

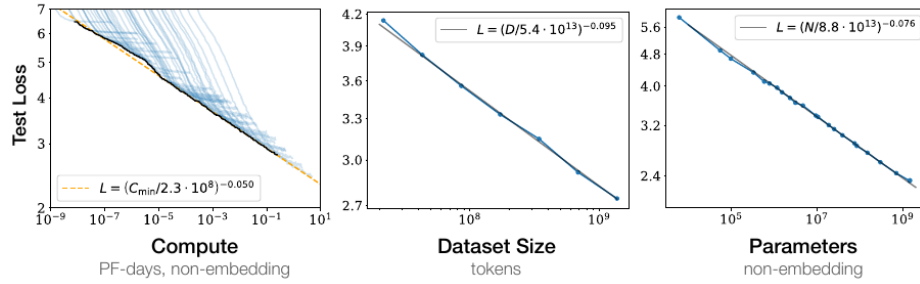


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Figure 1.2: Scaling-law trends from Kaplan et al. [14].

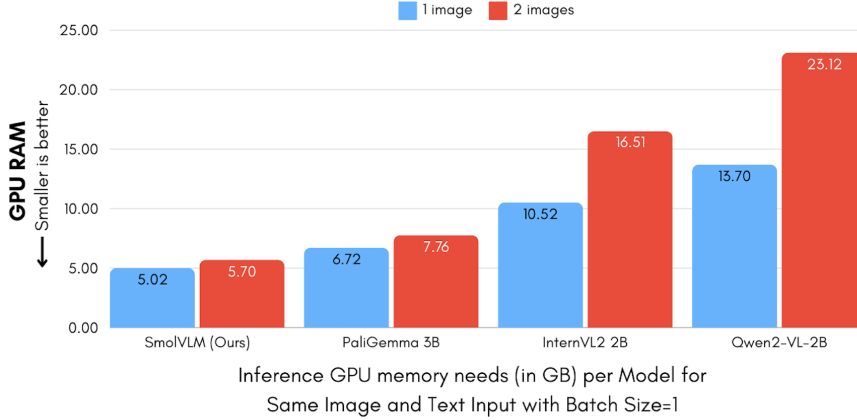


Figure 1.3: Inference GPU memory usage from the SmolVLM release blog [17, 18].

The vision side makes this constraint sharper in VLMs. VLMs insert visual embeddings or visual tokens into the multimodal context together with text, so the effective sequence length grows with image tokenization density. Since Transformer attention scales quadratically with sequence length, increasing the visual token budget quickly raises memory and latency [7, 16, 17]. For compact VLMs, this means that shrinking only the language backbone is not enough. The visual stack and the number of visual tokens also need to be controlled.

The most important regime here is **small models**, especially **sub-billion VLMs**. Models in the 1B to 3B range are already cheaper than large 7B to 70B systems, but they can still be heavy for edge deployment, memory-limited serving, or repeated evaluation across many checkpoints. Sub-billion students are a stricter target. They force the model to use its parameter budget and visual tokens more carefully, while still aiming to retain useful multimodal performance [12, 13, 17].

Deployment pressure is visible in Figure 1.3. The next comparison places compact VLM families on the parameter scale itself.

The issue is especially important for text-centric tasks such as scene-text reading, document understanding and chart reasoning [9–11].

Taken together, Figures 1.2, 1.3, 1.4 and 1.5 point to one conclusion: strong teachers benefit from scale, but the deployment target is still a much smaller student.

Knowledge distillation (KD) transfers knowledge from a large teacher to a compact student, often preserving strong performance at lower cost [22, 23].

Compact multimodal models can already be competitive (Figure 1.5), but closing the gap to stronger models still requires better transfer of knowledge.

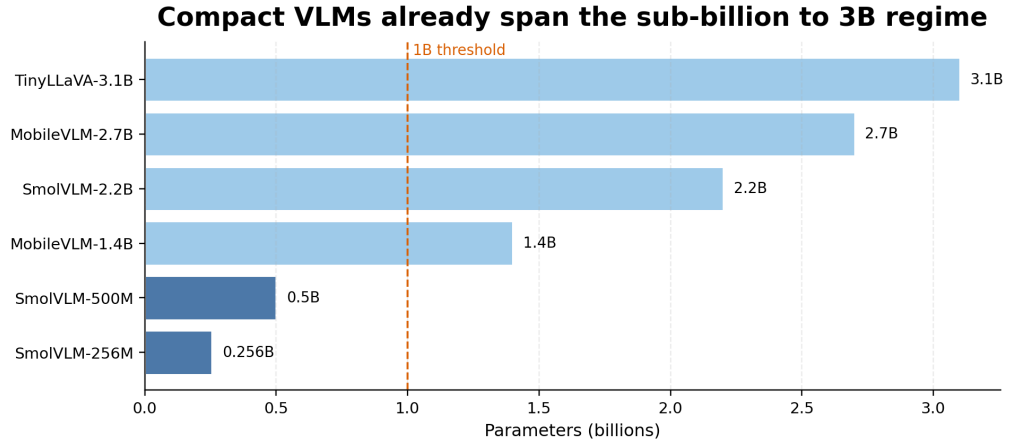


Figure 1.4: Representative compact VLM scales in the sub-billion to 3B range [12, 13, 17].

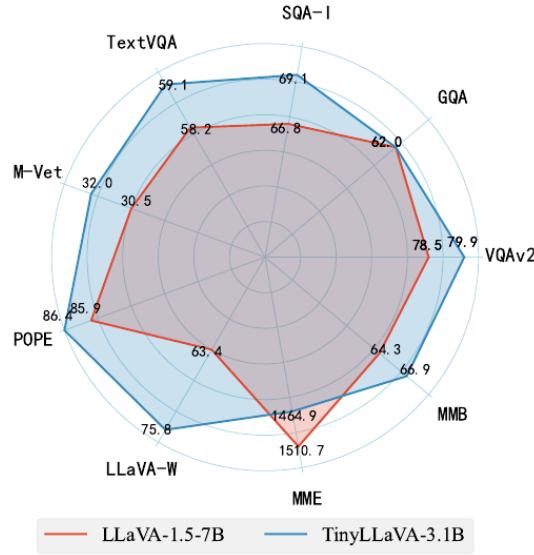


Figure 1.5: TinyLLaVA vs. LLaVA-1.5-7B benchmark comparison from [13].

Open-source frontier models are already close to proprietary leaders

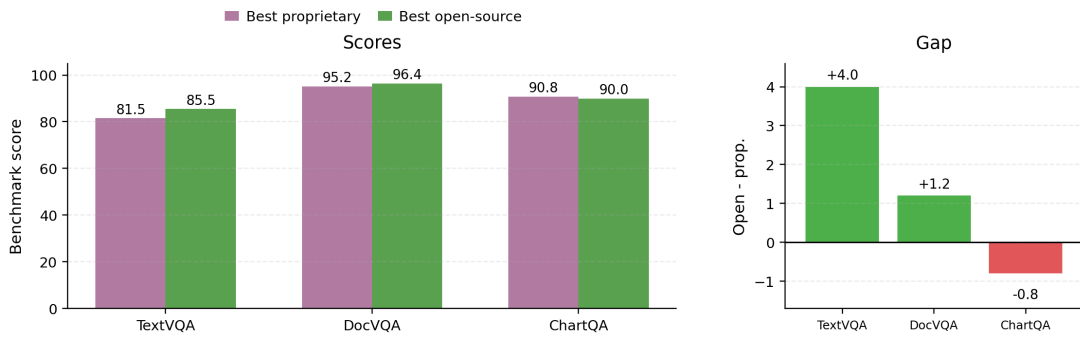


Figure 1.6: Representative proprietary vs. open-source frontier results on TextVQA, DocVQA and ChartQA [19–21].

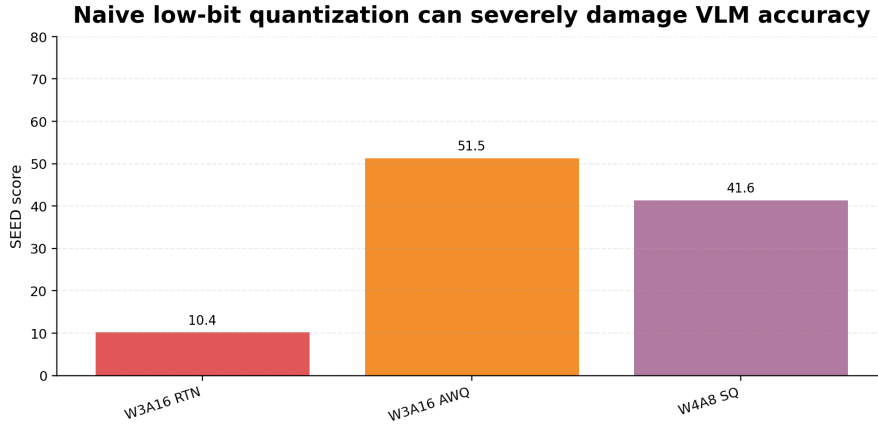


Figure 1.7: SEED accuracy of LLaVA-OneVision-7B under several low-bit quantization settings, reconstructed from MBQ Table 4 [32].

On these three benchmarks, the best open-source models are already close to the best proprietary models and in two cases they are ahead (Figure 1.6).

This makes white-box distillation especially attractive. In black-box KD, the student can only query an external teacher service and read its returned predictions. In white-box KD, the student can also use teacher logits and intermediate states [24, 25]. That extra access enables objectives such as the trie-Wasserstein KD loss used here, as well as prior baselines such as ULD and hidden-state matching, which are not available in a pure black-box setting [23, 26–28].

Another major competing compression path is quantization. Unlike distillation, quantization compresses an already trained model by reducing numerical precision. It can cut memory and improve inference speed substantially, so it is a serious practical alternative to KD [29–31].

In VLMs, however, naive low-bit quantization can hurt multimodal accuracy [31, 32]. For this reason, we treat quantization as a complementary deployment step, not a substitute for better transfer during training.

This observation is exactly why we focus on **sub-billion vision-language models**. They are small enough to make the efficiency constraint real, yet strong enough that improvements from better distillation remain meaningful [17].

In the multimodal setting, recent work has shown that distillation can transfer reasoning or representation knowledge into more efficient VLMs [8, 33, 34]. This is especially relevant on text-rich benchmarks such as TextVQA, DocVQA, and ChartQA [9–11].

The pattern in Figure 1.8 is the base case extended here to the heterogeneous multi-teacher setting.

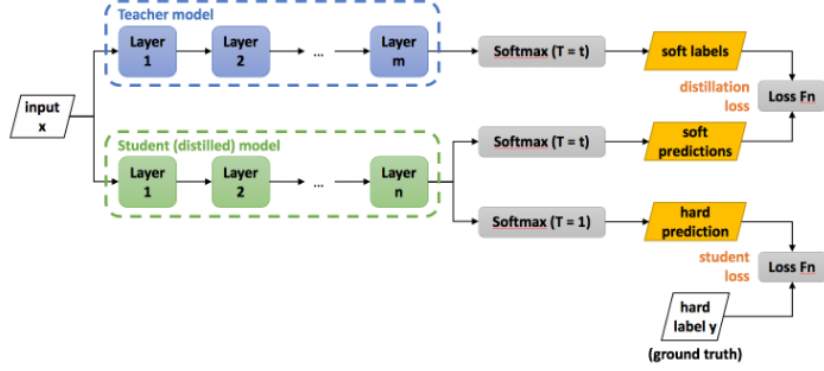


Figure 1.8: Generic teacher-student knowledge distillation [22, 35].

Since no single teacher excels across all multimodal tasks, *multi-teacher distillation* can provide richer supervision than a single-teacher pipeline [36–38].

In our setting, this is not just a matter of picking several strong teachers. The available teachers come from heterogeneous VLMs with different tokenizers, different output vocabularies and different sample-level strengths [26, 36, 37].

A naive uniform average can therefore mix teachers that help one sample but hurt another [23, 36, 37].

This motivates a selective multi-teacher framework over static teacher averaging.

We introduce Gradient-Routed Alignment with Capacity-aware Experts (GRACE), a selective multi-teacher distillation framework that routes teachers per sample, applies a trie-Wasserstein cross-tokenizer loss and refines teacher weights with gradient-based alignment. To the best of our knowledge, this is the first distillation framework to employ a shared tree-metric OT for heterogeneous multi-teacher VLM distillation. The framework tackles tokenizer mismatch, heterogeneous teacher specialization and post-routing instability in one recipe.

1.2 Related Work

The literature that matters most here is best grouped by transfer mechanism: matched-vocabulary logits distillation, cross-tokenizer distillation, representation matching and multi-teacher selection.

1.2.1 Compact VLM Distillation with Matched Output Spaces

Knowledge distillation is a standard way to compress large models into smaller students while retaining part of the teacher’s predictive structure [22, 23]. When teacher and student share the same tokenizer, the common choice is to match softened output distributions with a Kullback-Leibler (KL) divergence [22]. In multimodal systems, prior work has distilled large VLMs into smaller students, transferred reasoning behavior or improved compact vision-language models [8, 33, 34]. This line shows that compact VLMs benefit from teacher supervision, but it usually assumes one teacher or a matched output space. It therefore does not directly address heterogeneous teachers with different tokenizers.

Quantization is a separate compression line that reduces numerical precision after or during training [29–32]. It helps deployment, but it does not solve the transfer problem studied here.

1.2.2 Cross-Tokenizer and Transport-Based Distillation

Cross-tokenizer KD becomes necessary when teacher and student vocabularies do not match. ULD is a direct baseline in this setting because it compares sorted probability mass instead of token IDs [26]. More recent work also studies tokenizer-agnostic distillation through transport-based formulations [27, 28, 39].

The weakness of ranked-mass matching is that it discards relations between decoded strings. Two pairs with the same probability shape can therefore receive the same loss even when one pair shares a long prefix and the other does not. Tree-structured transport keeps that information by comparing mass on shared branches of a hierarchy [4, 27]. This leaves clear room for cross-tokenizer objectives that match teacher and student by mass on shared substructures, not by token ID.

1.2.3 Representation Matching Beyond Logits

Another line of work transfers intermediate representations instead of only final logits. Similarity measures such as CKA provide a tokenizer-independent way to compare hidden states across layers [40]. This makes representation matching attractive when teacher and student architectures differ. However, hidden-state alignment alone does not decide which teacher should dominate a sample and does not resolve conflicts among several teachers. It is therefore a useful baseline, but still incomplete for heterogeneous multi-teacher distillation.

1.2.4 Multi-Teacher Distillation and Adaptive Teacher Selection

Multi-teacher KD has been studied with fixed averages, stochastic teacher selection and meta-learned weighting rules [36–38]. These methods motivate the use of more than one teacher, but fixed or meta-learned weights can still mix useful and harmful teachers when teacher utility changes across samples.

Sparse routing and load balancing are well established in mixture-of-experts models [41–43]. Gradient-based filtering also appears in distillation through alignment-style objectives [44]. These ideas are relevant, but prior work does not jointly handle heterogeneous VLM teachers, tokenizer mismatch and post-routing instability in one compact-VLM setting. The remaining gap is a method that can select teachers per sample while staying stable under heterogeneous supervision.

1.2.5 Research Gap

Taken together, the literature leaves three gaps open: tokenizer mismatch across heterogeneous teachers and students, sample-dependent teacher utility and instability after adaptive routing. Those gaps define the technical problem studied in the following chapters.

1.3 Problem Statement and Research Objectives

The central problem is how to transfer multimodal understanding from large VLMs to compact students. The transfer must preserve task performance while reducing deployment cost at inference time.

We focus on text-rich multimodal tasks where the student must read image text, understand the question and perform basic reasoning [9–11].

In this setting, existing distillation pipelines often assume a single teacher, a shared tokenizer, or a fixed averaging rule across teachers. These assumptions are restrictive for heterogeneous multi-teacher VLM distillation [22, 26, 36, 37].

More specifically, the method should satisfy three requirements.

1. The KD loss should remain valid even when teacher and student tokenizers differ.
2. Teacher assignment should be *sample-dependent*, not fixed across the whole dataset.

3. The routing mechanism should remain sparse while suppressing routed teachers whose KD direction conflicts with the supervised objective. When needed, the framework should also support regularizers that discourage routing collapse.

Given a pool of N frozen teacher VLMs and a compact student, the goal is therefore to learn a distillation strategy that:

1. Remains valid across mismatched teacher and student vocabularies.
2. Routes teachers adaptively from student-side features through a learned student-conditioned gate.
3. Retains sparse top- k computation under capacity constraints.
4. Filters routed teachers when their KD update direction conflicts with the supervised objective, while keeping a fallback teacher when necessary.

Gradient-Routed Alignment with Capacity-aware Experts (GRACE) addresses these requirements.

GRACE is a selective multi-teacher distillation framework.

It combines a trie-Wasserstein cross-tokenizer KD loss, a student-conditioned teacher gate, top- k capacity-aware sparse routing and gradient-based GRACE score refinement after routing. The framework also supports routing fallback after capacity pruning, uniform fallback when score spread is too small and balancing and expert-bias updates [4, 26, 42–44].

We evaluate it on text-rich multimodal understanding tasks such as TextVQA, DocVQA and ChartQA [9–11].

Accordingly, we focus on cross-tokenizer multimodal distillation for compact students and evaluate GRACE on TextVQA, DocVQA and ChartQA, with analysis of routing behavior and representation matching.

1.4 Research Questions

The thesis is organized around three research questions.

1. **RQ1: Performance Comparison.** How does GRACE compare with same-family logits KD, cross-tokenizer ULD and other multi-teacher baselines on TextVQA, DocVQA and ChartQA?

2. **RQ2: Component Analysis.** What is the contribution of trie-Wasserstein KD, sparse routing and GRACE score refinement to the final performance of GRACE?
3. **RQ3: Efficiency Analysis.** How efficient is GRACE in terms of execution time and computational cost?

1.5 Thesis Contributions

The main contributions are:

1. We define a heterogeneous multi-teacher distillation setting for compact VLMs with mismatched teacher and student vocabularies on TextVQA, DocVQA and ChartQA.
2. We present Gradient-Routed Alignment with Capacity-aware Experts (GRACE), a selective multi-teacher distillation framework that combines trie-Wasserstein cross-tokenizer KD, a student-conditioned gate, sparse top- k routing and GRACE score refinement.
3. We show that this framework directly addresses the main practical failures of heterogeneous VLM distillation: tokenizer mismatch, sample-dependent teacher conflict and unstable teacher weighting after routing.
4. We establish consistent gains over fine-tuning and strong distillation baselines, supported by routing, ablation and representation analyses.

1.6 Thesis Organization

The remainder of the thesis is organized as follows. Chapter 2 summarizes the technical preliminaries, including Transformer-based VLMs, standard distillation notation, cross-tokenizer KD background, optimal transport and sparse routing concepts. Chapter 3 presents the proposed method, including the trie-Wasserstein KD loss, the student-conditioned router and the GRACE refinement mechanism. Chapter 4 describes the datasets, baselines, training configuration and evaluation protocol. Chapter 5 answers the three research questions through the main empirical results, ablations and efficiency analysis. The final chapter concludes the thesis and outlines the main limitations and future directions.

Chapter 2

Preliminaries

This chapter keeps only the technical preliminaries that Chapter 3 uses directly. It fixes the model notation, the generic distillation template, the cross-tokenizer background and the sparse-routing concepts behind GRACE. The broader motivation, related work and research gap already appear in Chapter 1, so they are not repeated here.

2.1 Transformer-Based Vision-Language Models

2.1.1 Transformer Basics

Modern vision-language models are built on the Transformer architecture [16]. For an input sequence, self-attention maps query, key and value matrices $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ to

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}, \quad (2.1)$$

where d_k is the key dimension. Multi-head attention runs this operation in parallel across several learned subspaces, then applies a feed-forward network and residual normalization.

For language modeling, the Transformer produces a sequence of hidden states and next-token logits. Here the student and teacher interact through their output logits because the distillation method operates at the token-distribution level [16, 22].

2.1.2 Vision Encoder and Multimodal Fusion

On the visual side, many multimodal systems use a Vision Transformer (ViT) [45], which converts an image into a sequence of patch embeddings and processes them with Transformer blocks. A connector module then fuses these visual tokens with text, using cross-attention, a Q-Former, or a learned projector [5–7].

We focus on compact generative VLMs in the same general family as SmolVLM and related projector-based models. Conceptually, we can write such a model as

$$\mathbf{z} = f_\theta(x), \quad (2.2)$$

where x is a multimodal input and $\mathbf{z}$ denotes the output logits used for autoregressive answer generation.

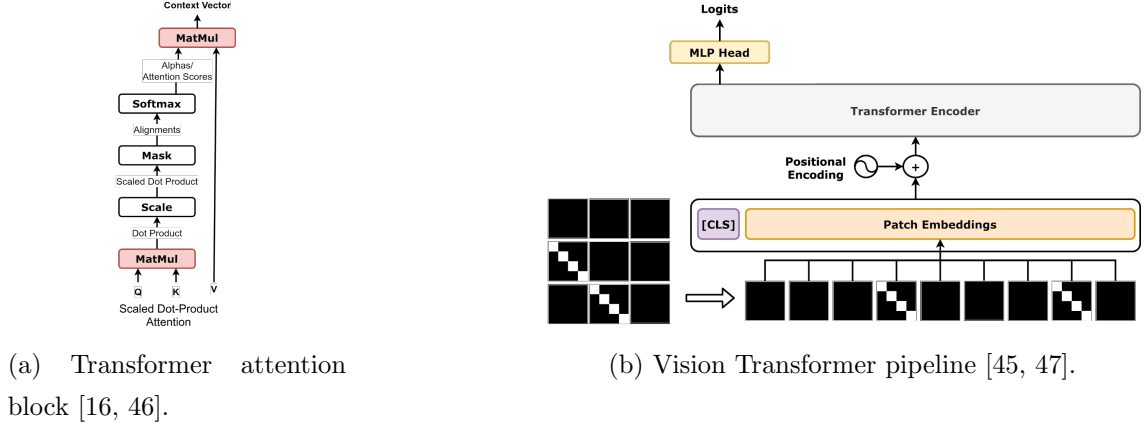


Figure 2.1: Transformer and ViT building blocks.

The basic Transformer and ViT blocks that later multimodal systems build on appear in Figures 2.1a and 2.1b.

The later experiments focus on text-rich multimodal understanding, where the model must read image text, interpret the question and generate a short answer sequence [9–11]. Modern VLM families fall into the groups in Figure 2.2.

2.2 Knowledge Distillation

2.2.1 Teacher-Student Learning

Knowledge distillation transfers information from a stronger teacher model to a smaller student model [22, 23]. Let $\mathcal{L}_{CE}$ denote the supervised cross-entropy loss of the student. A generic distillation objective takes the form

$$\mathcal{L} = \lambda_{CE}\mathcal{L}_{CE} + \lambda_{KD}\mathcal{L}_{KD}, \quad (2.3)$$

where $\mathcal{L}_{KD}$ is a teacher-guided loss. In later chapters, GRACE instantiates this template with a routed trie-Wasserstein term plus small router regularizers.

In token-level distillation, the teacher and student are compared on their output logits, not only on hard labels. This allows the student to learn richer distributional structure, often called *dark knowledge* [22].

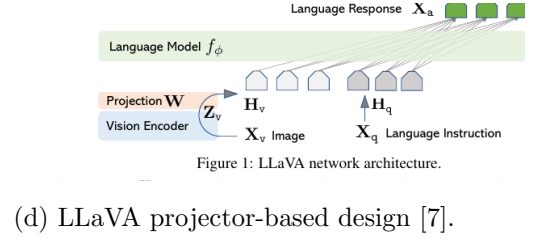
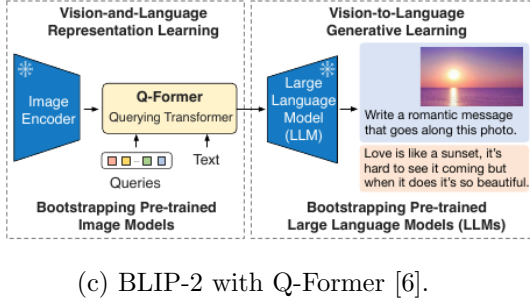
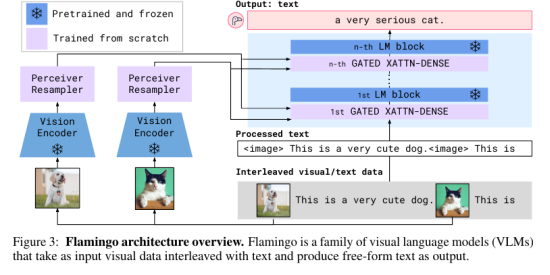
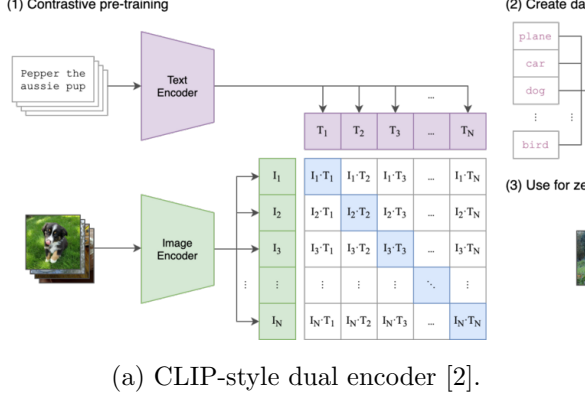


Figure 2.2: Representative architecture families of modern vision-language models.

2.2.2 Temperature-Scaled Output Distributions

Let $\mathbf{z}_s$ and $\mathbf{z}_t$ denote student and teacher logits. With temperatures τ_s and τ_t , the corresponding distributions are

$$\mathbf{p}_s = \text{softmax}\left(\frac{\mathbf{z}_s}{\tau_s}\right), \quad \mathbf{p}_t = \text{softmax}\left(\frac{\mathbf{z}_t}{\tau_t}\right). \quad (2.4)$$

Many classical KD methods compare $\mathbf{p}_s$ and $\mathbf{p}_t$ using divergences such as forward KL or Jensen-Shannon divergence. However, such approaches usually assume that teacher and student share the same vocabulary [22, 26, 48].

2.3 Cross-Tokenizer Distillation Background

2.3.1 ULD as a Prior Baseline

When the teacher and student vocabularies differ, KD losses defined on a shared output space are not directly applicable. Universal Logit Distillation (ULD) is one prior remedy because it avoids direct token-ID matching [26].

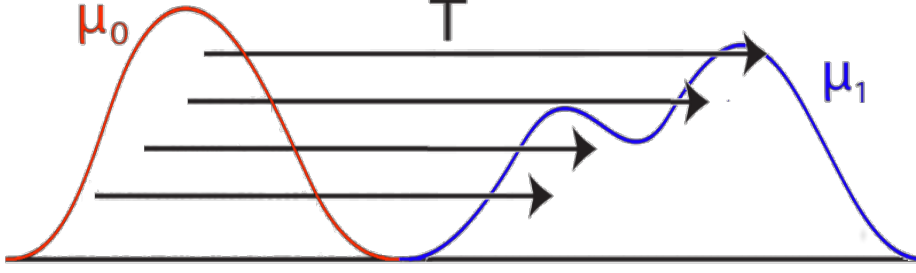


Figure 2.3: Continuous displacement view of optimal transport [3, 4].

Given student logits $\mathbf{z}_s \in \mathbb{R}^{V_s}$ and teacher logits $\mathbf{z}_t \in \mathbb{R}^{V_t}$, we first compute their temperature-scaled probability distributions $\mathbf{p}_s$ and $\mathbf{p}_t$. We then sort both vectors in descending order, pad the smaller vocabulary with zeros and compare the sorted masses with an L_1 distance:

$$\mathcal{L}_{ULD}(\mathbf{z}_s, \mathbf{z}_t) = \|\pi(\mathbf{p}_s) - \pi(\mathbf{p}_t)\|_1, \quad (2.5)$$

where $\pi(\cdot)$ denotes sorting in descending order after zero-padding.

The key advantage of ULD is that it compares distributions by ranked probability mass instead of token identity. This makes it directly applicable to cross-tokenizer teacher-student distillation [26]. In this thesis, ULD serves as a cross-tokenizer baseline.

2.3.2 Optimal Transport and Tree-Structured Mass Matching

Optimal transport compares two probability measures by the minimum cost required to move mass from one distribution to the other [4]. For discrete source weights $\mathbf{a}$, target weights $\mathbf{b}$ and transport cost matrix $\mathbf{C}$, the corresponding optimization problem can be written as

$$W(\mathbf{a}, \mathbf{b}) = \min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{P}, \mathbf{C} \rangle, \quad \mathcal{U}(\mathbf{a}, \mathbf{b}) = \{\mathbf{P} \geq 0 \mid \mathbf{P}\mathbf{1} = \mathbf{a}, \mathbf{P}^\top \mathbf{1} = \mathbf{b}\}. \quad (2.6)$$

Here, $\mathbf{P}$ is a transport plan that specifies how much mass is moved from each source bin to each target bin.

The continuous geometric view of OT in Figure 2.3 carries over to the discrete case through a transport plan $\mathbf{P}$, and later to the tree case through edgewise subtree mass.

This viewpoint also motivates tree-structured transport. When teacher and student have different vocabularies, matching probabilities by token index is not meaningful. What matters instead is how probability mass sits on a shared structure [4, 26].

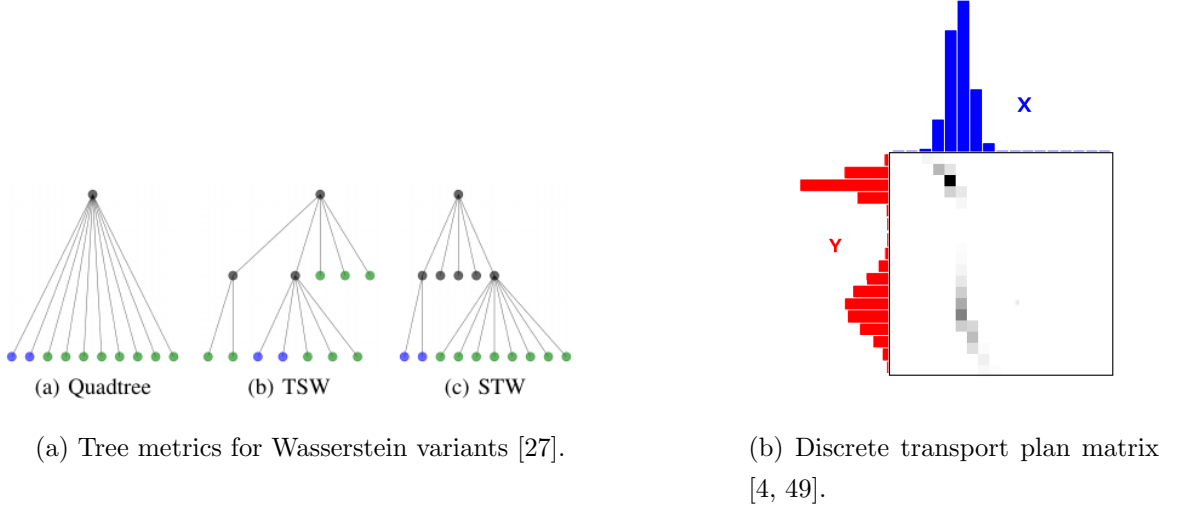


Figure 2.4: Tree-structured and matrix views of discrete optimal transport.

Tree-Wasserstein distances make that idea explicit by placing probability mass on a tree metric and summing weighted subtree imbalances [4, 27]. For a rooted tree $T = (V, E)$ with edge weights $w(e)$, one convenient form is

$$W_T(\mu, \nu) = \sum_{e \in E} w(e) |\mu(\Gamma(e)) - \nu(\Gamma(e))|, \quad (2.7)$$

where $\Gamma(e)$ denotes the subtree below edge e . The key point is that, on a tree metric, transport decomposes into edgewise subtree-mass imbalance instead of a dense source-target coupling [27, 39]. This is computationally attractive because the comparison follows the tree structure directly and it is conceptually natural when tokens share prefixes or other hierarchical relations.

Tree-sliced OT extends this idea by averaging Wasserstein distances over tree metrics and shows that tree metrics preserve the geometric view of OT while admitting fast closed-form computations [39]. Here the relevant tree is not an abstract hierarchy but the shared UTF-8 byte trie induced by teacher and student decoded single-token strings.

That perspective is closer to the trie-Wasserstein loss used later in Chapter 3: the method maps tokens to paths on a shared byte trie and compares mass on edges instead of token index.

Together, Figures 2.3, 2.4b and 2.4a cover the three views used later in the thesis: geometric mass movement, discrete transport plans and tree-structured mass matching. Chapter 3 instantiates the third view with a shared byte trie and a sparse tree-Wasserstein loss.

2.3.3 Answer-Token Distillation

For multimodal question answering, distillation should focus on answer tokens, not prompt tokens [7].

In our implementation, the code computes KD only on supervised answer positions after masking ignored tokens, optionally removing EOS and truncating student and teacher answer spans to a common length.

In Chapter 3, these sets are denoted by Ω_b^{CE} for CE and $\Omega_{b,i}^{KD}$ for teacher-specific KD.

This gives a token-aligned distillation signal while remaining compatible with different teacher and student tokenizers.

2.4 Multi-Teacher Distillation

2.4.1 Per-Teacher KD Loss

Let $\mathcal{T} = \{T_1, \dots, T_N\}$ be a pool of frozen teachers. For sample b , teacher T_i produces a per-teacher distillation loss $\mathcal{L}_{KD}^{(b,i)}$. In principle, one could average these losses uniformly,

$$\mathcal{L}_{KD}^{(b)} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{KD}^{(b,i)}, \quad (2.8)$$

but this assumes every teacher is equally useful for every sample.

In practice, different teachers may provide conflicting guidance. Some teachers may help the supervised objective on a given sample, while others may push the student in a different direction [44]. This motivates a selective routing mechanism that decides, for each sample, which teachers should be active.

2.4.2 Learned Routing and Load Balancing

Recent sparse expert models route each sample or token through only a subset of available experts. This improves efficiency and allows specialized computation [41, 42, 50].

The same routing idea is useful in multi-teacher distillation. Instead of averaging all teachers equally, a learned router can propose teacher weights from student-side features [36–38].

Let $\mathbf{h}^{(b)}$ denote a pooled student representation for sample b . A router network $G(\cdot)$ produces gate logits

$$\mathbf{r}^{(b)} = G(\mathbf{h}^{(b)}), \quad \tilde{\mathbf{r}}^{(b)} = \text{AdjustedScores}(\mathbf{r}^{(b)}), \quad \mathbf{p}^{(b)} = \text{softmax}(\tilde{\mathbf{r}}^{(b)}), \quad (2.9)$$

where $p_i^{(b)}$ is the initial routing weight for teacher T_i . In the current implementation, $\tilde{\mathbf{r}}^{(b)}$ includes expert bias, optional training noise and router-temperature scaling before the routing softmax.

In sparse routing, these weights are typically refined by additional constraints:

1. **Top- k routing**, which keeps only the highest-scoring teachers.
2. **Capacity constraints**, which prevent one teacher from receiving too many assignments.
3. **Load balancing**, which discourages routing collapse onto a small subset of teachers.

Following the general MoE literature, one can summarize the balancing signal by a soft load $\bar{p}_i$ and a hard assignment load $\bar{q}_i$ after sparse routing, capacity control and fallback and define an auxiliary balancing objective of the form

$$\mathcal{L}_{bal} = N \sum_{i=1}^N \bar{p}_i \bar{q}_i. \quad (2.10)$$

This auxiliary term does not replace the KD objective; instead, it regularizes the router so that different teachers remain usable during training.

In GRACE, the code implements the router as a small student-conditioned MLP gate on top of pooled student hidden states. It then applies softmax weights, top- k sparsification and capacity filtering, with a short dense warmup before sparse routing starts. The full framework also supports optional balancing, entropy regularization and expert-bias updates during training.

When enabled, the expert bias term nudges the router toward under-used teachers in subsequent batches.

This makes the routing stage closer to sparse-expert training than to a static teacher average [41–43].

The next figure adds the specialization view. Sparse routing does not only save compute, it also lets different experts focus on different inputs.

The sparse-routing ideas that motivate the teacher gate in GRACE appear in Figures 2.5 and 2.6.

Prior work also uses gradient similarity in distillation as a signal for whether teacher supervision helps or conflicts with the student objective [44]. Chapter 3 builds on that idea after routing, where GRACE scores only the teachers that remain active for a sample.

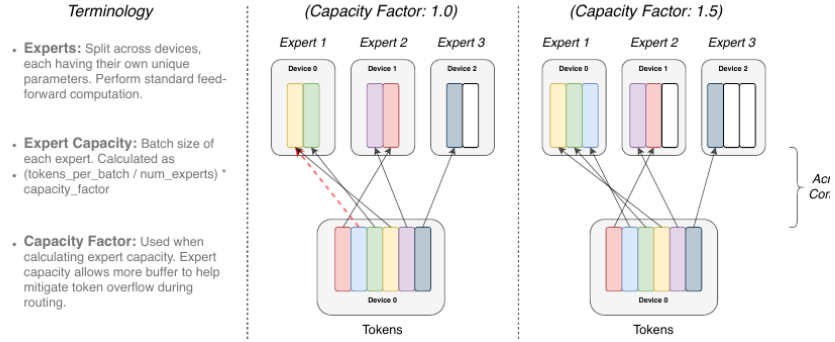


Figure 2.5: Sparse routing with expert capacity constraints, adapted from Switch Transformers [42].

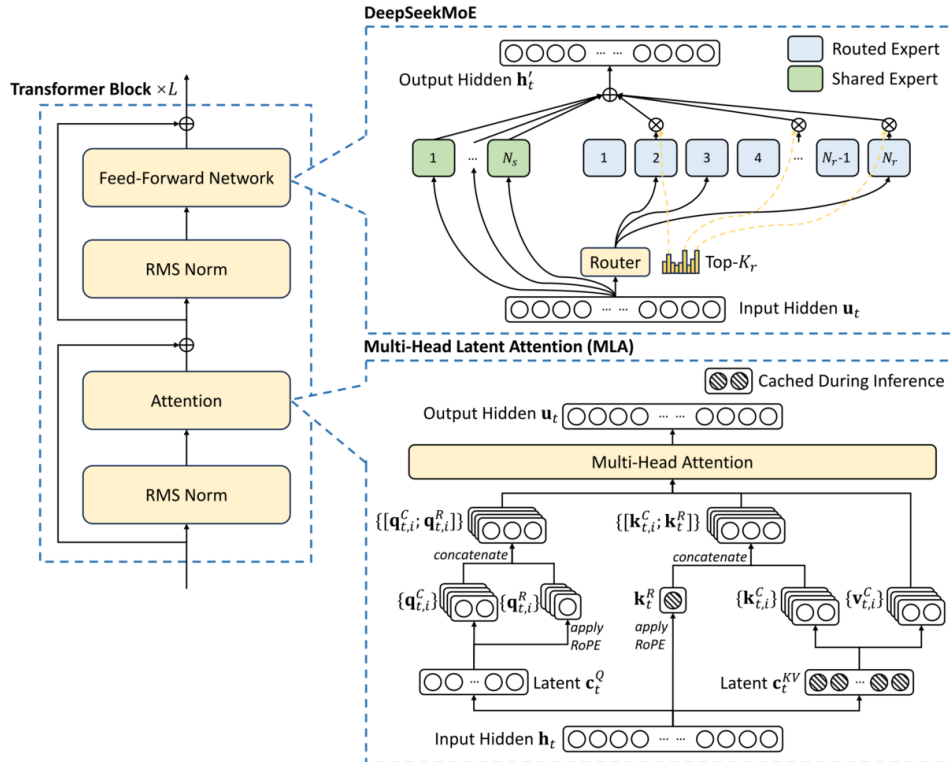


Figure 2.6: Sparse routing and expert specialization in a mixture-of-experts architecture [50, 51].

Chapter 3

Proposed Method

This chapter presents the full GRACE method. It first formulates the multi-teacher distillation problem, then defines the trie-Wasserstein KD loss, the student-conditioned router and the GRACE score-refinement stage. It then gives a compact notation summary, a one-page intuition and the full training step before the later sections expand each component. It closes with the final training objective.

3.1 Method Overview and Problem Formulation

We consider text-rich multimodal understanding tasks that require reading image text, understanding the question and generating an answer sequence [9–11]. The training set is

$$\mathcal{D} = \{(x_b, y_b)\}_{b=1}^B, \quad (3.1)$$

where x_b is a multimodal input and y_b is the target answer sequence.

Let S_θ denote the student VLM with trainable parameters θ and let

$$\mathcal{T} = \{T_1, T_2, \dots, T_N\} \quad (3.2)$$

be a pool of frozen teacher VLMs. For each sample b , the student produces logits $\mathbf{z}_s^{(b)}$ and teacher T_i produces teacher logits $\mathbf{z}_t^{(b,i)}$.

The goal is not to pre-select a fixed teacher subset before training. Instead, we train the student with *Gradient-Routed Alignment with Capacity-aware Experts (GRACE)*, a routed multi-teacher distillation framework that refines sparse teacher assignments with gradient-based scores.

GRACE has five components:

1. A supervised student objective $\mathcal{L}_{CE}$.
2. A per-teacher trie-Wasserstein objective $\mathcal{L}_{KD}^{(b,i)}$.
3. A student-conditioned MLP gate.
4. Top- k routing, capacity control and router regularization.
5. GRACE score refinement based on pooled gradient cosine similarity.

At a high level, the gate first proposes dense teacher weights from pooled student hidden states. A short hard-routing warmup keeps all teachers available; after that, top- k and capacity constraints make the routing sparse. Each routed teacher then contributes a candidate KD loss and a GRACE score that measures directional agreement with the supervised CE update.

Before GRACE warmup, the method uses uniform KD weights over the teachers that remain after routing. After warmup, GRACE smooths the scores with an EMA, converts them to a score softmax, blends them with the routed weights, and falls back to a uniform distribution only when the score spread is too small to justify a strong preference.

The final objective combines supervised CE, routed KD and small router regularizers. Its full form appears in Section 3.5.

This design is closer in spirit to selective gradient-based distillation [44] and sparse expert routing [41]. Large-scale routed Transformer systems such as Switch Transformers show the same need for sparse assignment and overflow control [42].

Here we adapt that idea to the multi-teacher multimodal setting with token-level logit distillation losses.

The notation table, one-page intuition and early algorithm below fix the global picture first. The detailed subsections then define the trie-Wasserstein loss, routed teacher selection and GRACE score refinement in detail.

3.1.1 Notation Summary

The method follows four stages:

1. Run the student to obtain $(\mathbf{z}_s, \mathbf{H}_s, \mathcal{L}_{CE})$.
2. Compute gate logits $\mathbf{r}$, adjusted routing scores $\tilde{\mathbf{r}}$, soft routing weights $\mathbf{p}$ and route teachers under top- k and capacity constraints.
3. Compute routed per-teacher trie-Wasserstein losses $\mathcal{L}_{KD}^{(b,i)}$.
4. Compute pooled gradient-alignment scores, smooth them over time, blend them with the routed weights and apply fallback when needed.

This four-stage view is the shortest way to read GRACE end to end: route first, score routed teachers next, then form the effective KD weights that enter the final loss.

Symbol	Meaning
$\mathcal{D} = \{(x_b, y_b)\}_{b=1}^B$	Multimodal samples and target answer sequences.
$S_\theta, \mathcal{T} = \{T_i\}_{i=1}^N$	Student VLM and frozen teacher set.
$\mathbf{z}_s^{(b)}, \mathbf{z}_t^{(b,i)}$	Student and teacher logits for sample b .
$\mathbf{H}_s, \mathbf{h}$	Student hidden states and pooled routing feature.
$\mathbf{r}, \tilde{\mathbf{r}}, \mathbf{p}$	Router logits, adjusted scores and soft router weights.
$\mathcal{R}_b, \hat{\mathbf{p}}^{(b)}, \mathbf{w}^{(b)}$	Routed teacher set, routed weights and final KD weights.
$\Omega_b^{CE}, \Omega_{b,i}^{KD}$	Supervised CE positions and matched KD positions.
Route	Sparse teacher routing with top- k , capacity and fallback.
U	Uniform weights over the routed teacher set.
Blend	Mixing routed weights with GRACE score weights.
EMA	Exponential moving average for GRACE scores.

Table 3.1: Main symbols used in Chapter 3.

3.1.2 Training Step

Here RouterScores, GradientProxies, ScoreWeights and BatchKD are shorthands for the exact definitions in Equation 3.12, the later gradient-proxy equations, the GRACE score softmax and Equation 3.34. The exact warmup schedule, fallback rule and final teacher weights remain in the equations below.

3.2 Trie-Wasserstein Cross-Tokenizer Distillation

3.2.1 Supervised Answer Tokens

We apply distillation only on supervised answer tokens, not on prompt tokens. For sample b , let Ω_b^{CE} denote the student positions whose labels are not ignored. For each teacher T_i , let $\Omega_{b,i}^{KD}$ denote the matched answer positions after:

1. Masking ignored labels on both student and teacher sides.
2. Optionally removing EOS from the student or teacher answer span.
3. Truncating both spans to a common length.

This produces one supervised student set for CE and one matched student-teacher set for each KD term.

Algorithm 1 GRACE Training Step

Input: mini-batch $\{(x_b, y_b)\}_{b=1}^B$, frozen teachers $\{T_i\}_{i=1}^N$

State: student S_θ , teacher gate state, GRACE EMA state

Hyperparameters: $k, C_t, \rho_h, \rho_g, \tau_g, \beta, \epsilon, \alpha, \lambda_{bal}, \lambda_{ent}, \lambda_z, \eta$

Phase 1: Student Forward

1: $(\mathbf{z}_s, \mathbf{H}_s, \mathcal{L}_{CE}) \leftarrow S_\theta(x)$

2: $\mathbf{h} \leftarrow \text{Pool}(\mathbf{H}_s)$

Phase 2: Teacher Routing

3: $(\tilde{\mathbf{r}}, \mathbf{p}) \leftarrow \text{RouterScores}(\mathbf{h})$

4: $k_t \leftarrow \text{WarmupTopK}(t; N, k, \rho_h T)$

5: **for** each sample $b = 1, \dots, B$ **do**

6: $(\hat{\mathbf{p}}^{(b)}, \mathcal{R}_b) \leftarrow \text{Route}(\tilde{\mathbf{r}}^{(b)}; k_t, C_t)$

7: **end for**

Phase 3: KD Loss Computation

8: **for** each sample $b = 1, \dots, B$ **do**

9: **for** each routed teacher $i \in \mathcal{R}_b$ **do**

10: $\mathbf{z}_t^{(b,i)} \leftarrow T_i(x_b)$

11: $\mathcal{L}_{KD}^{(b,i)} \leftarrow \mathcal{L}_{TW}(\mathbf{z}_s^{(b)}, \mathbf{z}_t^{(b,i)}; \Omega_{b,i}^{KD})$

12: **end for**

13: **end for**

Phase 4: GRACE Refinement and Update

14: **if** $t < \rho_g T$ **then**

15: $w^{(b)} \leftarrow U(\mathcal{R}_b)$ for all b

16: **else**

17: $(\tilde{\mathbf{g}}_{CE}^{(b)}, \{\tilde{\mathbf{g}}_i^{(b)}\}) \leftarrow \text{GradientProxies}(\mathbf{z}_s, \{\mathbf{z}_t^{(b,i)}\})$ for all b

18: $\mathbf{s}^{(b)} \leftarrow \text{Align}(\tilde{\mathbf{g}}_{CE}^{(b)}, \{\tilde{\mathbf{g}}_i^{(b)}\})$ for all b

19: $\hat{\mathbf{s}}^{(b)} \leftarrow \text{EMA}(\mathbf{s}^{(b)})$ for all b

20: $\mathbf{q}^{(b)} \leftarrow \text{ScoreWeights}(\hat{\mathbf{s}}^{(b)} | \mathcal{R}_b)$ for all b

21: $\mathbf{w}^{(b)} \leftarrow \text{Blend}(\hat{\mathbf{p}}^{(b)}, \mathbf{q}^{(b)})$ for all b

22: apply uniform fallback when $\Delta^{(b)} < \epsilon$

23: **end if**

24: $\mathcal{L}_{KD} \leftarrow \text{BatchKD}(\{\mathbf{w}^{(b)}, \mathcal{R}_b, \mathcal{L}_{KD}^{(b,i)}\}_{b=1}^B)$

25: $\mathcal{L} \leftarrow \mathcal{L}_{CE} + \alpha \mathcal{L}_{KD} + \lambda_{bal} \mathcal{L}_{bal} + \lambda_{ent} \mathcal{L}_{ent} + \lambda_z \mathcal{L}_z$

26: $\theta \leftarrow \text{Update}(\theta, \mathcal{L}, \eta)$

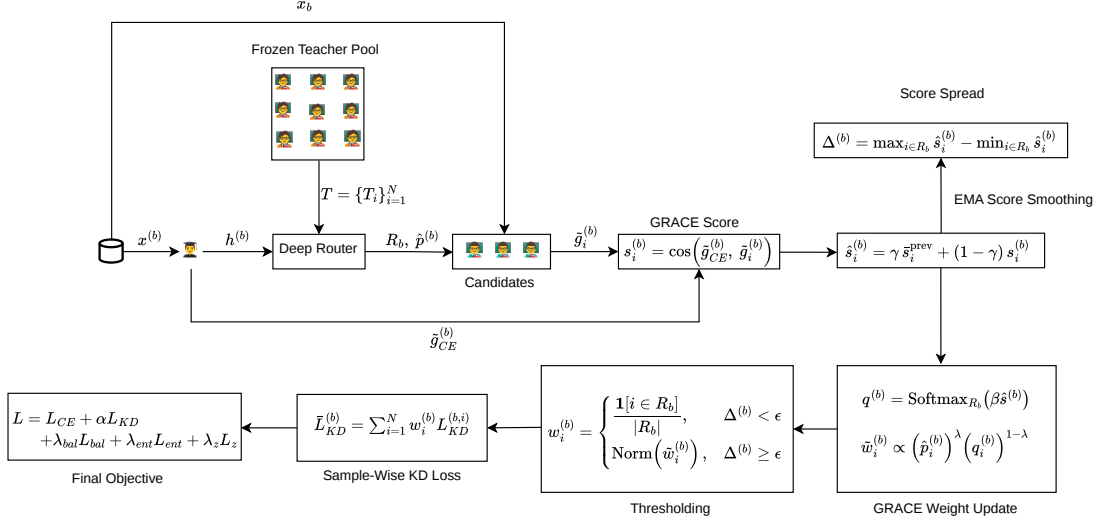


Figure 3.1: Architecture view of GRACE.

3.2.2 Shared Byte Trie

For each sample b , token position $j \in \Omega_{b,i}^{KD}$ and teacher T_i , let $\mathbf{z}_{s,j}^{(b)} \in \mathbb{R}^{V_s}$ be the student logits and $\mathbf{z}_{t,j}^{(b,i)} \in \mathbb{R}^{V_i}$ be the teacher logits. The student and teacher probability distributions are

$$\mathbf{p}_{s,j}^{(b)} = \text{softmax}\left(\frac{\mathbf{z}_{s,j}^{(b)}}{\tau_s}\right), \quad \mathbf{p}_{t,j}^{(b,i)} = \text{softmax}\left(\frac{\mathbf{z}_{t,j}^{(b,i)}}{\tau_t}\right), \quad (3.3)$$

where τ_s and τ_t are the student and teacher temperatures.

The implementation does not compare these distributions by token ID or sorted rank. Instead, the method decodes each single-token vocabulary entry to a string, converts that string to UTF-8 bytes, appends an EOS sentinel, and inserts the resulting byte path into one shared trie. If e is a trie edge at depth $d(e)$, its transport weight is

$$w(e) = \begin{cases} \rho^{d(e)-1}, & e \neq \text{TAIL}, \\ 1, & e = \text{TAIL}, \end{cases} \quad (3.4)$$

where $0 < \rho < 1$ is the edge-decay factor. Shared prefixes therefore receive matching early edges, while deeper trie disagreements are discounted. The synthetic TAIL edge is handled separately with weight 1 [4, 27].

Different tokenizers may stop at different depths, but they still share the same byte-prefix edges when their decoded token strings are related. The trie therefore **matches probability mass on shared byte paths instead of token IDs**. The example in Figure 3.2 uses the pair document/receipt and doc/rece.

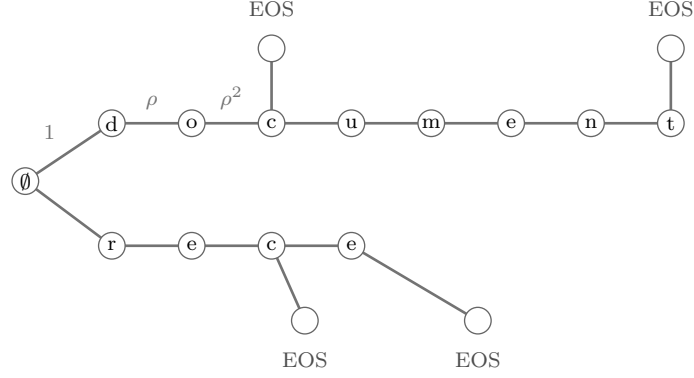


Figure 3.2: Shared-trie view of cross-tokenizer matching for `document/receipt` and `doc/rece`.

In Figure 3.2, shorter teacher pieces terminate earlier with EOS, while longer student pieces continue deeper in the trie, so the mismatch becomes structural instead of purely symbolic token matching.

3.2.3 Sparse Top- k Support and Tail Edge

For efficiency, the loss keeps only the top- k_{TW} non-ignored student and teacher tokens at each supervised position. Let $\mathcal{K}_{s,j}^{(b)}$ and $\mathcal{K}_{t,j}^{(b,i)}$ be those retained token sets. The code then collapses the remaining probability mass into a dedicated TAIL edge:

$$r_{s,j}^{(b)} = 1 - \sum_{v \in \mathcal{K}_{s,j}^{(b)}} p_{s,j}^{(b)}(v), \quad r_{t,j}^{(b,i)} = 1 - \sum_{v \in \mathcal{K}_{t,j}^{(b,i)}} p_{t,j}^{(b,i)}(v). \quad (3.5)$$

For any active trie edge e , the retained subtree masses are

$$m_{s,j}^{(b)}(e) = \sum_{v \in \mathcal{K}_{s,j}^{(b)} : e \in \text{path}(v)} p_{s,j}^{(b)}(v) + \mathbf{1}[e = \text{TAIL}] r_{s,j}^{(b)}, \quad (3.6)$$

$$m_{t,j}^{(b,i)}(e) = \sum_{v \in \mathcal{K}_{t,j}^{(b,i)} : e \in \text{path}(v)} p_{t,j}^{(b,i)}(v) + \mathbf{1}[e = \text{TAIL}] r_{t,j}^{(b,i)}. \quad (3.7)$$

The token-level trie-Wasserstein loss is then

$$\ell_{TW}(\mathbf{z}_{s,j}^{(b)}, \mathbf{z}_{t,j}^{(b,i)}) = \sum_{e \in \mathcal{E}_{act,j}^{(b,i)}} w(e) \left| m_{s,j}^{(b)}(e) - m_{t,j}^{(b,i)}(e) \right|, \quad (3.8)$$

where $\mathcal{E}_{act,j}^{(b,i)}$ contains the active student edges, active teacher edges and the TAIL edge.

The per-sample, per-teacher KD loss remains the mean over supervised answer tokens:

$$\mathcal{L}_{KD}^{(b,i)} = \frac{1}{|\Omega_{b,i}^{KD}|} \sum_{j \in \Omega_{b,i}^{KD}} \ell_{TW}(\mathbf{z}_{s,j}^{(b)}, \mathbf{z}_{t,j}^{(b,i)}). \quad (3.9)$$

Complexity Analysis. The shared byte trie is built once with time and memory cost $O(V_s \bar{L}_s + V_t \bar{L}_t)$, where V_s and V_t are the student and teacher vocabulary sizes and $\bar{L}_s$ and $\bar{L}_t$ are the average byte-path lengths. For one aligned token position, the sparse trie-Wasserstein loss costs $O(V_s \log K + V_t \log K + A \log A)$, where K is the top- k budget and A is the number of active trie edges.

Runtime memory is $O(E + P_s + P_t + A)$, where E is the number of trie edges and P_s and P_t are the stored student and teacher path-index arrays. Unlike dense OT, the implementation never materializes a $|V_s| \times |V_t|$ transport matrix.

Compared with the ULD baseline from Chapter 2, this loss compares mass on shared trie edges instead of ranked logits. It keeps the cross-tokenizer goal, but it uses a structured transport view that is closer to the actual tokenizer geometry, while the top- k +TAIL approximation keeps the cost bounded by the active support.

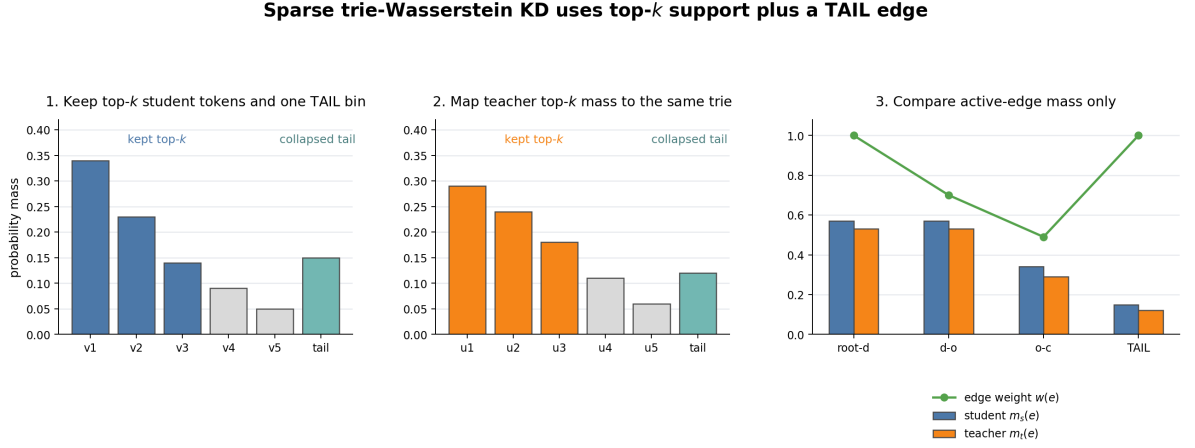


Figure 3.3: Sparse trie-Wasserstein support in the implementation.

The sparse view explains why the loss remains practical for compact students: the code never builds a dense transport matrix and the TAIL edge captures the residual low-probability mass in one term.

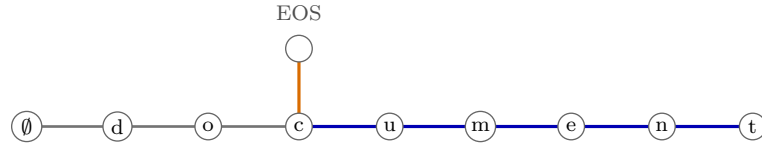


Figure 3.4: Trie split for **document** and **doc**.

Teacher / student top token	$\mathcal{L}_{ULD}$	ℓ_{TW}
doc / document	0.20	0.60
abc / def	0.20	3.29

Table 3.2: Illustrative comparison between ULD and trie-Wasserstein under the same ranked masses. ULD assigns the same loss to both rows, while trie-Wasserstein gives lower loss to the pair with a shared byte prefix.

The split in Figure 3.4 links the equations to one case. The shared prefix contributes little because both sides place mass on the same early edges. The main loss appears after c : the teacher sends mass to EOS, while the student sends mass to the continuation path, so the token-level term grows on the first mismatched edges.

Table 3.2 gives one small comparison against the ULD baseline. In both rows, the ranked masses are the same: the teacher places 0.8 on its top token and 0.2 on the residual tail, while the student places 0.9 on its top token and 0.1 on the residual tail. With $\rho = 0.5$ and $\text{top-}k_{TW} = 1$, ULD returns an identical value in both cases because it only compares sorted mass. Trie-Wasserstein gives a much smaller loss to `doc` vs `document` than to `abc` vs `def` because the first pair shares a long byte prefix.

3.3 Learned Teacher Gate and Load Balancing

3.3.1 Student-Conditioned Router

When multiple teachers are available, the method first builds a student-conditioned router instead of averaging them uniformly.

Let $\mathbf{H}_s^{(b)}$ denote the student hidden states captured at the routing source layer for sample b . We pool these hidden states over supervised answer positions to obtain a routing feature

$$\mathbf{h}^{(b)} = \text{Pool}(\mathbf{H}_s^{(b)}) . \quad (3.10)$$

In code, this pool is the mean over supervised answer positions. If a sample has no supervised answer token, the gate falls back to the positions selected by the attention mask.

A student-conditioned MLP router $G(\cdot)$ then produces gate logits

$$\mathbf{r}^{(b)} = G(\mathbf{h}^{(b)}) . \quad (3.11)$$

The code then forms routing scores

$$\tilde{\mathbf{r}}^{(b)} = \frac{\mathbf{r}^{(b)} + \mathbf{b} + \boldsymbol{\xi}^{(b)}}{\tau_g}, \quad \mathbf{p}^{(b)} = \text{softmax}(\tilde{\mathbf{r}}^{(b)}) , \quad (3.12)$$

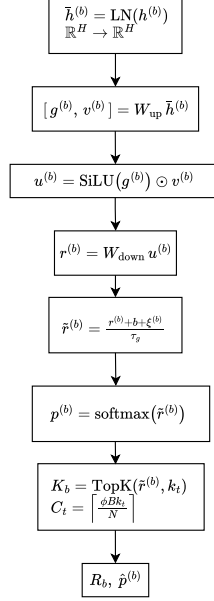


Figure 3.5: Student-conditioned router sketch in GRACE.

where $\mathbf{b}$ is the expert-bias vector, τ_g is the router temperature and $\xi^{(b)}$ is Gaussian router noise during training. Here $p_i^{(b)}$ is the initial gate weight for teacher T_i .

The gate stack in Figure 3.5 applies LayerNorm $\rightarrow$ up projection $\rightarrow$ split into gate and value branches $\rightarrow$ SiLU(gate) $\odot$ value $\rightarrow$ down projection, then applies bias, top- k , capacity control, fallback and balance updates to the resulting teacher logits.

3.3.2 Top-k, Capacity and Fallback

Let k_t denote the effective top- k value at training step t . The router is made sparse by applying top- k routing to the routing scores $\tilde{r}_i^{(b)}$. Let $\mathcal{K}_b = \text{TopK}(\tilde{\mathbf{r}}^{(b)}, k_t)$ be the candidate teacher set for sample b . To prevent one teacher from receiving too many assignments, each teacher is additionally limited by a capacity

$$C_t = \left\lceil \phi \frac{B k_t}{N} \right\rceil, \quad (3.13)$$

where B is the mini-batch size, N is the number of teachers, and $\phi > 0$ is the capacity factor.

Let $m_i^{(b)} \in \{0, 1\}$ denote the resulting assignment mask after top- k selection, capacity pruning and fallback. The constrained gate weights are

$$\hat{p}_i^{(b)} = \frac{m_i^{(b)} p_i^{(b)}}{\sum_{j=1}^N m_j^{(b)} p_j^{(b)}}. \quad (3.14)$$

If capacity pruning leaves a sample without any assigned teacher, the algorithm falls back to the teacher with the largest bias-adjusted routing score $\tilde{r}_i^{(b)}$.

This ensures that at least one teacher remains available before GRACE score refinement.

The implementation also uses a short hard-routing warmup. Let ρ_h be the hard-routing warmup ratio. During the first $\rho_h T$ steps, the router uses

$$k_t = N, \quad (3.15)$$

so all teachers remain available. After that point, the router switches to the configured sparse value $k_t = k$.

3.3.3 Balancing Loss and Expert Bias Update

The full GRACE framework regularizes the gate with a balancing loss that uses both soft routing weights and hard assignments.

Define the soft load

$$\bar{p}_i = \frac{1}{B} \sum_{b=1}^B p_i^{(b)} \quad (3.16)$$

and the hard assignment load

$$\bar{q}_i = \frac{\sum_{b=1}^B m_i^{(b)}}{\sum_{b=1}^B \sum_{j=1}^N m_j^{(b)}}, \quad (3.17)$$

The balancing objective is

$$\mathcal{L}_{bal} = N \sum_{i=1}^N \bar{p}_i \bar{q}_i. \quad (3.18)$$

The expert bias is updated from the per-batch assignment load ℓ_i to discourage teacher overuse:

$$b_i \leftarrow b_i - \eta_b \left(\ell_i - \frac{1}{N} \sum_{j=1}^N \ell_j \right), \quad (3.19)$$

where η_b is a small bias-update rate.

The code also adds two small auxiliary router penalties:

- A negative-entropy term

$$\mathcal{L}_{ent} = -\frac{1}{B} \sum_{b=1}^B H(\mathbf{p}^{(b)}), \quad (3.20)$$

which discourages premature routing collapse.

- A router z -loss

$$\mathcal{L}_z = \frac{1}{B} \sum_{b=1}^B \left(\log \sum_{i=1}^N e^{r_i^{(b)}} \right)^2, \quad (3.21)$$

which keeps router logits numerically well behaved.

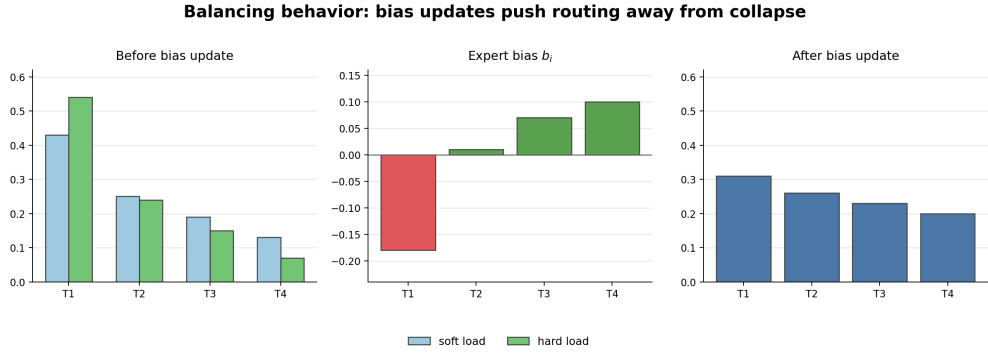


Figure 3.6: Bias-feedback behavior in GRACE under expert-bias updates.

The bias update is isolated in Figure 3.6. The routing example in Figure 3.7 is inspired by the routing-overflow and load-balancing behavior used in sparse expert models [42, 43]. The dense gate may prefer a small subset of teachers, but top- k selection, capacity constraints and fallback convert these scores into a sparse assignment pattern with explicit load statistics.

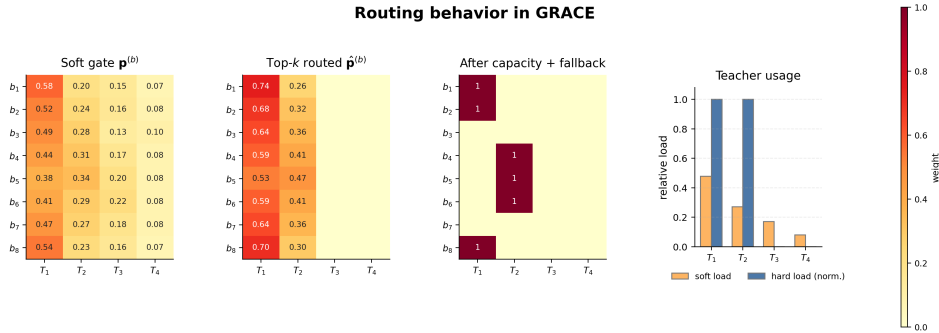


Figure 3.7: Routing dynamics in GRACE under top- k , capacity and fallback.

3.4 GRACE Score Refinement

3.4.1 Gradient Proxies

For each sample b , let $\mathcal{L}_{CE}^{(b)}$ be the student cross-entropy loss on supervised answer tokens. We define the pooled CE logit-gradient proxy as

$$\tilde{\mathbf{g}}_{CE}^{(b)} = \frac{1}{|\Omega_b^{CE}|} \sum_{j \in \Omega_b^{CE}} \nabla_{\mathbf{z}_{s,j}^{(b)}} \mathcal{L}_{CE}^{(b)}. \quad (3.22)$$

In implementation, the code computes this gradient in closed form from the student probabilities and labels, which avoids a second backward pass through the full model.

For each teacher T_i , the pooled KD logit-gradient proxy is

$$\tilde{\mathbf{g}}_i^{(b)} = \frac{1}{|\Omega_b^{CE}|} \sum_{j \in \Omega_{b,i}^{KD}} \nabla_{\mathbf{z}_{s,j}^{(b)}} \mathcal{L}_{KD}^{(b,i)}. \quad (3.23)$$

The code computes this proxy directly from the selected distillation loss. In the default setting here, $\mathcal{L}_{KD}^{(b,i)}$ is the trie-Wasserstein loss in Equation 3.9. The KD sum uses the matched student-teacher positions $\Omega_{b,i}^{KD}$, but the normalization still uses the student CE count $|\Omega_b^{CE}|$.

3.4.2 GRACE Scores and Logged Active Flags

For each sample b and teacher T_i , we measure directional agreement between supervised learning and KD by cosine similarity:

$$s_i^{(b)} = \cos\left(\tilde{\mathbf{g}}_{CE}^{(b)}, \tilde{\mathbf{g}}_i^{(b)}\right). \quad (3.24)$$

The CE proxy averages over Ω_b^{CE} , while the KD proxy sums over $\Omega_{b,i}^{KD}$ and normalizes by $|\Omega_b^{CE}|$. This matches the current implementation.

The implementation also records a thresholded activity flag

$$a_i^{(b)} = \mathbf{1}[s_i^{(b)} > \delta], \quad (3.25)$$

where δ is a logging threshold. These flags are useful for monitoring, but the GRACE weight update itself uses the continuous scores $s_i^{(b)}$. Figure 3.8 gives the directional picture behind this cosine score.

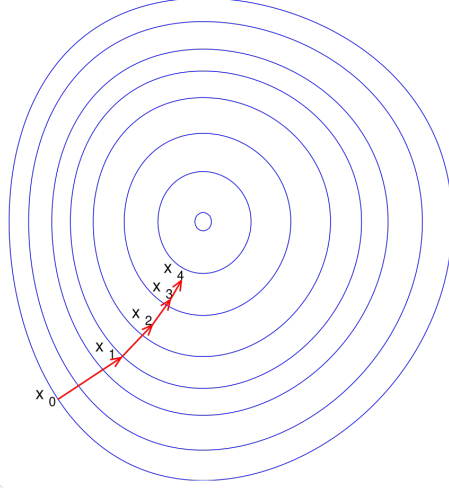


Figure 3.8: Directional view of gradient descent on a contour surface [52].

3.4.3 Warmup, Score Smoothing and Gradient Weights

GRACE refinement starts only after a warmup phase. Let $\rho_g \in [0, 1]$ denote the GRACE warmup ratio with respect to the total number of training steps. During the first $\rho_g T$ steps, the code uses a uniform KD distribution over the teachers that remain after routing:

$$u_i^{(b)} = \frac{\mathbf{1}[\hat{p}_i^{(b)} > 0]}{\sum_{j=1}^N \mathbf{1}[\hat{p}_j^{(b)} > 0]}. \quad (3.26)$$

After warmup, GRACE refines the routed teachers with a score-smoothed gradient distribution. Let $\bar{s}_i^{\text{prev}}$ be the running exponential moving average (EMA) of past batch means for teacher T_i and let γ be the EMA decay. The per-sample smoothed score is

$$\hat{s}_i^{(b)} = \gamma \bar{s}_i^{\text{prev}} + (1 - \gamma) s_i^{(b)}. \quad (3.27)$$

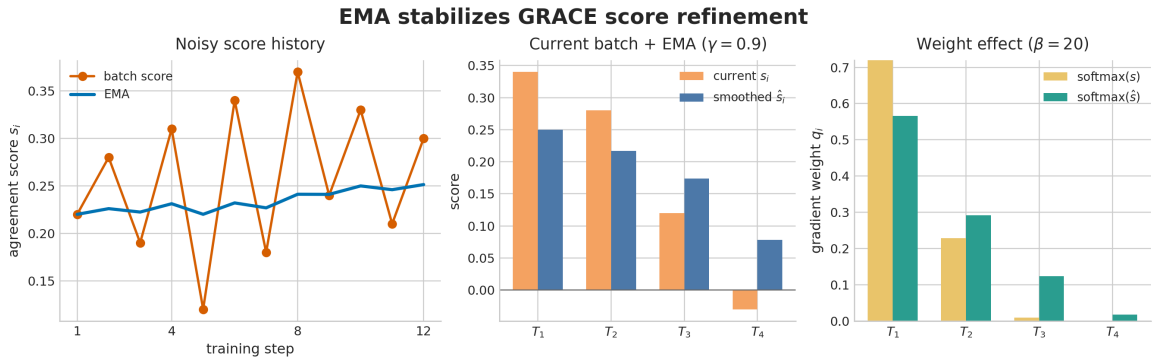


Figure 3.9: EMA smoothing in GRACE.

Raw per-batch scores are noisy, while smoothed scores produce more stable teacher weights, a point illustrated by Figure 3.9.

Let $\mathcal{R}_b = \{i \mid \hat{p}_i^{(b)} > 0\}$ be the routed teacher set. GRACE converts the smoothed scores into a gradient-weight vector

$$q_i^{(b)} = \frac{\exp(\beta \hat{s}_i^{(b)}) \mathbf{1}[i \in \mathcal{R}_b]}{\sum_{j=1}^N \exp(\beta \hat{s}_j^{(b)}) \mathbf{1}[j \in \mathcal{R}_b]}, \quad (3.28)$$

where β controls how sharply the highest-scoring teacher is preferred.

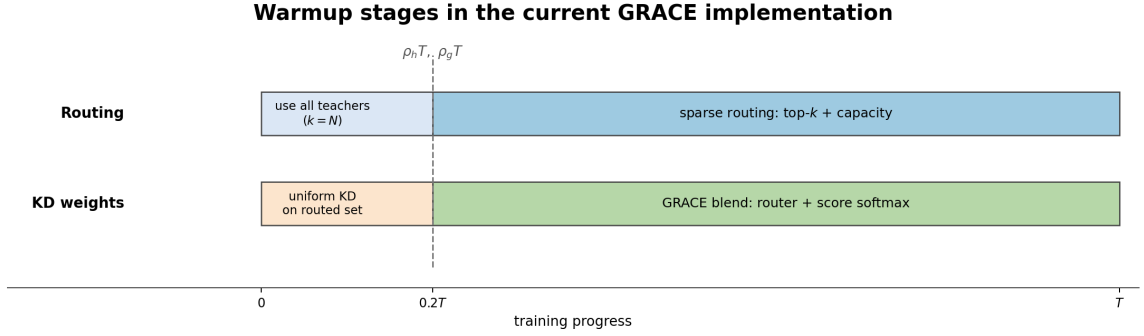


Figure 3.10: Warmup schedule in the implementation of GRACE.

The activation points for hard routing and GRACE weighting appear in Figure 3.10.

3.4.4 Router-Gradient Blend and Uniform Fallback

After warmup, GRACE combines the routed weights and the gradient weights by a geometric blend. Let $\lambda \in [0, 1]$ be the router-blend exponent. The unnormalized blended weights are

$$\tilde{w}_i^{(b)} \propto \left(\hat{p}_i^{(b)}\right)^\lambda \left(q_i^{(b)}\right)^{1-\lambda}, \quad (3.29)$$

followed by normalization over the routed teachers:

$$w_i^{(b)} = \frac{\tilde{w}_i^{(b)} \mathbf{1}[i \in \mathcal{R}_b]}{\sum_{j=1}^N \tilde{w}_j^{(b)} \mathbf{1}[j \in \mathcal{R}_b]}. \quad (3.30)$$

The implementation uses a uniform fallback only when the smoothed scores are too close to justify a strong preference. Define the score spread

$$\Delta^{(b)} = \max_{i \in \mathcal{R}_b} \hat{s}_i^{(b)} - \min_{i \in \mathcal{R}_b} \hat{s}_i^{(b)}. \quad (3.31)$$

If $\Delta^{(b)} < \epsilon$, then GRACE falls back to

$$w_i^{(b)} = \frac{\mathbf{1}[i \in \mathcal{R}_b]}{|\mathcal{R}_b|}. \quad (3.32)$$

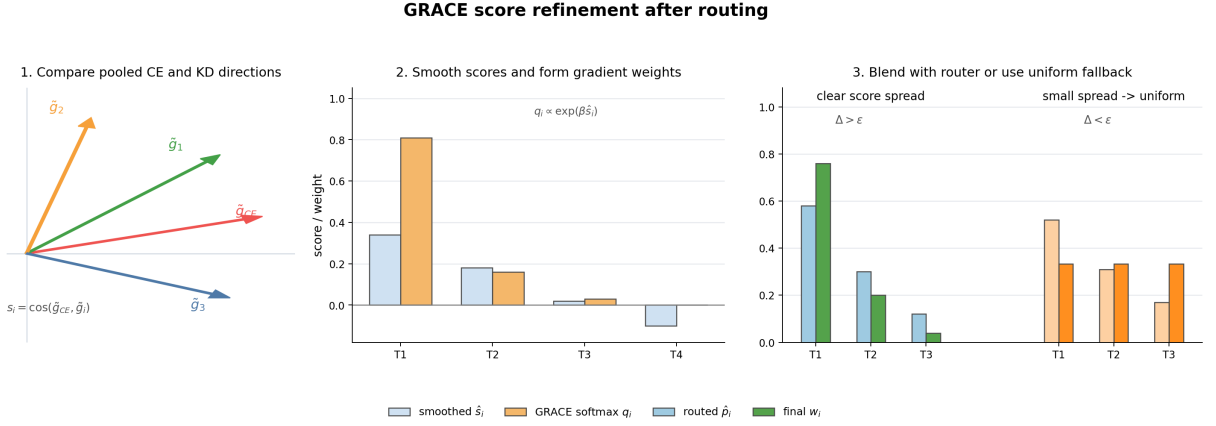


Figure 3.11: GRACE score refinement in GRACE.

This keeps KD stable when the routed teachers have nearly indistinguishable GRACE scores.

Figures 3.11 and 3.12 separate the score update from its resulting weighting patterns.

The weight view in Figure 3.12 recasts this rule as a weight update. The routed gate weights are first proposed by the student-conditioned router, then refined by GRACE scores and finally converted into the effective KD weights used in Equation 3.33.

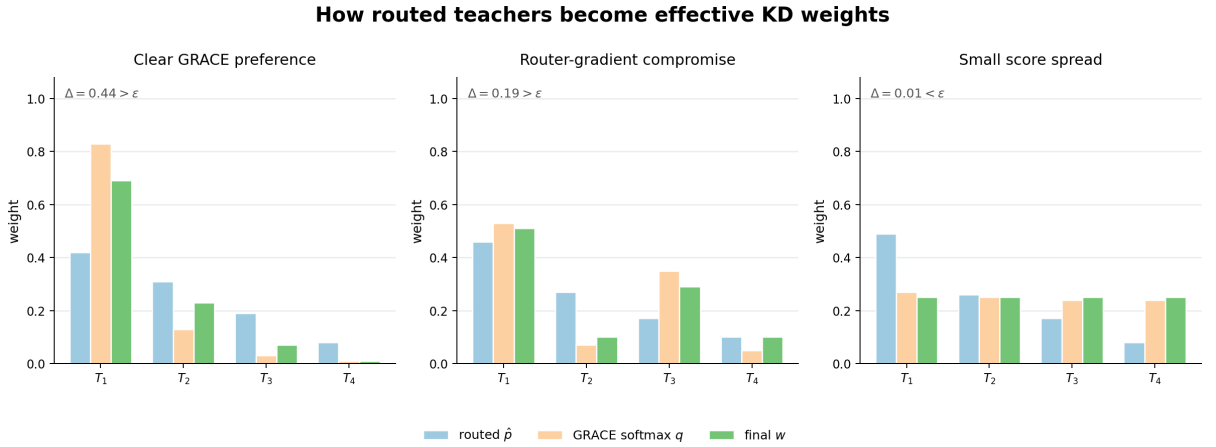


Figure 3.12: Representative GRACE weighting cases in GRACE.

The sample-wise path in Figure 3.13 compresses that update into one sample-wise path, from gate softmax to routed weights, GRACE softmax and final effective KD weights.

One teacher set can lead to different final weights on different samples, as Figure 3.14 shows.

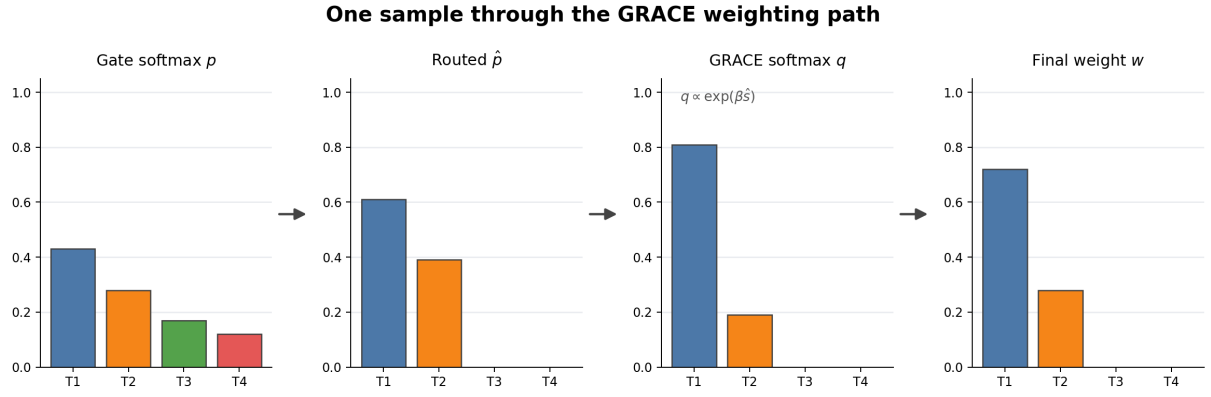


Figure 3.13: Sample-wise weight refinement in GRACE.

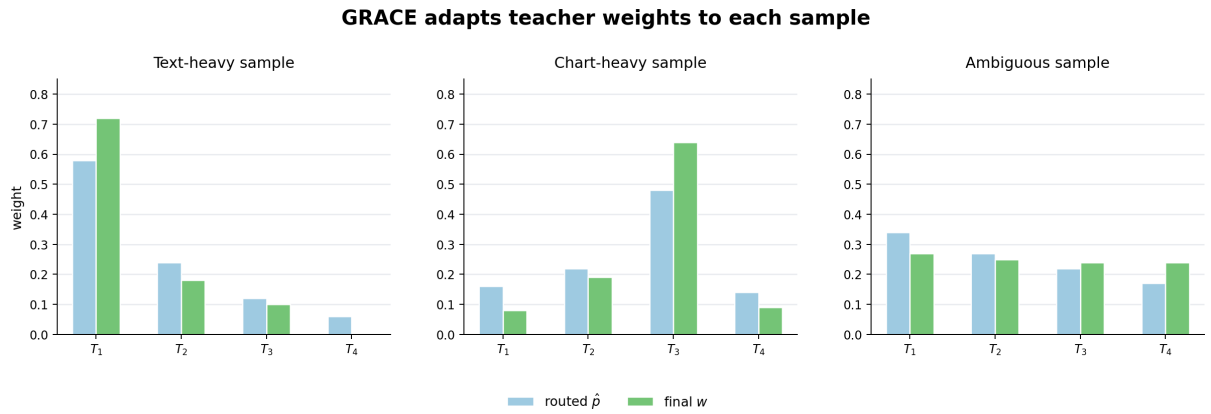


Figure 3.14: Sample-specific weighting in GRACE.

3.4.5 Sample-Wise KD Aggregation

The routed KD loss for sample b is the weighted aggregation

$$\bar{\mathcal{L}}_{KD}^{(b)} = \sum_{i=1}^N w_i^{(b)} \mathcal{L}_{KD}^{(b,i)}. \quad (3.33)$$

We then average these sample losses over the batch:

$$\mathcal{L}_{KD} = \frac{1}{B} \sum_{b=1}^B \bar{\mathcal{L}}_{KD}^{(b)}. \quad (3.34)$$

This rule has two stages: the gate defines the routed teacher set and GRACE refines those routed teachers into the final KD weights.

3.5 Final Objective

The final optimization objective is

$$\mathcal{L} = \mathcal{L}_{CE} + \alpha \mathcal{L}_{KD} + \lambda_{bal} \mathcal{L}_{bal} + \lambda_{ent} \mathcal{L}_{ent} + \lambda_z \mathcal{L}_z, \quad (3.35)$$

where $\mathcal{L}_{CE}$ is the standard supervised next-token cross-entropy loss and $\mathcal{L}_{KD}$ is defined in Equation 3.34. The balance term $\mathcal{L}_{bal}$ is defined in Equation 3.18; the entropy term $\mathcal{L}_{ent}$ and router z -loss $\mathcal{L}_z$ are small penalties on the teacher gate.

Equation 3.35 keeps the student anchored to ground-truth supervision while adding routed KD and small router regularizers. Compared with static multi-teacher averaging, this objective makes the teacher contribution adaptive at the sample level. Compared with fixed teacher-subset selection, it avoids committing to one subset for the whole run [36, 37, 44].

Together with the early algorithmic summary, Equation 3.35 completes the formal training recipe used in the experiments.

Chapter 4

Evaluation Methodology

This chapter describes how the methods are evaluated. It introduces the students, datasets, baselines and evaluation protocol used in the comparison. It also records the implementation details needed to reproduce the active training path, but keeps those engineering notes separate from the main experimental protocol.

4.1 Experimental Setup

All experiments follow a shared evaluation protocol for fair comparison across methods.

The study focuses on two compact student models, **SmolVLM-500M** and **SmolVLM-256M**, evaluated on text-rich multimodal benchmarks [17].

We evaluate all models with **VLM-Eval-Kit** [53]. In brief, the toolkit works as follows:

1. Load a benchmark split and apply its prompt template or input format.
2. Run model inference through a unified model wrapper interface.
3. Post-process raw generations into normalized answers.
4. Compute dataset-specific scores (e.g., VQA Accuracy, ANLS, Relaxed Accuracy) and aggregate final results.

This unified pipeline ensures a consistent evaluation procedure across all compared models.

4.2 Datasets

We evaluate three text-centric VQA benchmarks: **TextVQA**, **DocVQA** and **ChartQA**.

The goal is to measure whether the model can **read textual content in images, understand questions and perform basic reasoning** across different visual domains [9–11].

We chose these datasets because they require optical character recognition (OCR)-heavy understanding, cover complementary visual domains and include retrieval, layout understanding and numerical reasoning [9–11].

4.2.1 Dataset Descriptions

TextVQA [9]: evaluates question answering on natural images containing scene text. It focuses on reading text in-the-wild and combining it with question understanding. *Sample question*: “What word is written on the stop sign?” *Sample answer*: “No word is visible on this sign.”



Figure 4.1: Example image for a TextVQA-style question [54].

DocVQA [10]: evaluates VQA on real document images (scans/photos), where layout, paragraphs, tables and forms are important. It emphasizes document structure understanding in addition to OCR. *Sample question*: “What is the invoice number shown in this document?” *Sample answer*: “4572.”

ChartQA [11]: evaluates question answering over charts. It tests chart reading plus numerical and logical reasoning to derive the final answer. *Sample question*: “Which year has the highest sales value in the chart?” *Sample answer*: “The second (middle) bar has the highest value.”

The evaluation covers the three visual domains in Figures 4.1, 4.2 and 4.3.

4.2.2 Dataset Size and Splits

Table 4.1 lists the split sizes used throughout the evaluation chapter.



Figure 4.2: Example document image for a DocVQA-style question [55].

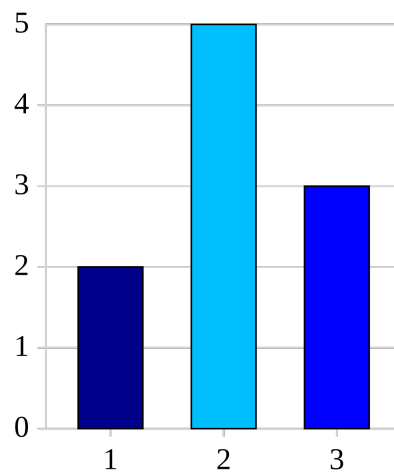


Figure 4.3: Example chart image for a ChartQA-style question [56].

Dataset	Train	Val	Test	Total
TextVQA	34,602	5,000	5,734	45,336
DocVQA	39,463	5,349	5,188	50,000
ChartQA-H (human)	7,398	960	1,250	9,608
ChartQA-M (machine)	20,901	960	1,250	23,111
ChartQA (H+M, combined)	28,299	1,920	2,500	32,719

Table 4.1: Train, validation and test split sizes for TextVQA, DocVQA and ChartQA [9–11]. ChartQA is also broken down into its human and machine-generated subsets.

4.3 Training Configuration

Unless otherwise stated, the main distillation experiments use a shared training configuration so that differences in performance can be attributed to the distillation strategy, not unrelated optimization choices.

We run all training jobs on a single high-memory **NVIDIA H200** graphics processing unit (GPU) on **Modal** or on comparable single-GPU **Vast.ai** instances, depending on availability. A typical full H200 run takes about 7 hours and costs about 21 USD on Vast.ai, excluding the separate cost of offline teacher-logit generation and remote storage. Distillation training uses **DeepSpeed ZeRO Stage 2**, **Brain Floating Point 16 (bf16)** mixed precision, **TensorFloat-32 (TF32)** matrix multiplication and gradient checkpointing to control memory use while keeping the student fully trainable and all teachers frozen [57, 58].

Before training, we convert all benchmark samples into a unified **LLaVA-style multi-modal instruction format** [7]. Each example is stored as an image plus a short user-assistant dialogue, so TextVQA, DocVQA and ChartQA can share the same data loader, prompt template and answer supervision interface.

Listing 4.1 shows the structure used in our preprocessing pipeline.

```

1 {
2   "image": "sample_000123.jpg",
3   "conversations": [
4     {"from": "human", "value": "<image>\nWhat is the invoice number?"},
5     {"from": "gpt", "value": "4572"}
6   ]
7 }
```

Listing 4.1: Representative LLaVA-style sample used after dataset conversion.

Table 4.2 summarizes the main optimization and routing settings used in the active launch path. The engineering details of teacher-logit storage and remote caching are deferred to Section 4.6, because they support reproducibility but are not part of the fair-comparison protocol itself.

Table 4.2: Main optimization and routing settings for the active three-teacher DocVQA distillation launch path. These values define the fair-comparison protocol used for the main training recipe, while broader engineering details are deferred to Section 4.6.

Setting	Value
Student model	HuggingFaceTB/SmolVLM-500M-Instruct
	Qwen/Qwen2.5-VL-3B-Instruct
Teacher models	Qwen/Qwen2-VL-2B-Instruct
	ibm-granite/granite-vision-3.1-2b-preview
Teacher weighting strategy	routing (GRACE path)
Distillation loss	Trie-Wasserstein
Trie-Wasserstein rho	0.7
Trie-Wasserstein top- k	64
Training dataset	DocVQA
Validation split	validation
Hardware and runtime	Single-GPU runs on Modal (H200) or Vast.ai
Typical run cost	About 7 hours on H200, about 21 USD on Vast.ai
Distributed training	DeepSpeed ZeRO-2
Optimizer	adamw_torch
Adam betas	(0.9, 0.98)
Adam epsilon	1×10^{-7}
Weight decay	0.0
Numerical precision	bf16 enabled, half precision (fp16) disabled, TF32 enabled
FlashAttention-2	Enabled
Training data format	LLaVA-style multimodal instruction JSON
Max sequence length	16384
Training epochs	2
Per-device batch size	80
Gradient accumulation steps	1
Effective batch size	80
Base learning rate	1×10^{-5}
Vision learning rate	2×10^{-6}
Connector learning rate	1×10^{-5}
Learning-rate schedule	Cosine decay with warmup ratio 0.03
Gradient checkpointing	Enabled

Continued on the next page.

Table 4.2 continued from the previous page.

Setting	Value
Frozen modules	None
Dataloader workers	4
Logging and checkpointing	Log every 5 steps; save every 150 steps
Student/teacher temperature	$\tau_s = \tau_t = 1.0$
KD weight	$\alpha = 0.5$
Top- k routing	$k = 2$
Capacity factor	$\phi = 1.25$
Teacher-gate temperature	1.5
Teacher-gate noise std	0.01
Balance weight	$\lambda_{bal} = 0.01$
Gate entropy weight	$\lambda_{ent} = 10^{-3}$
Router z -loss weight	$\lambda_z = 10^{-3}$
Bias-update rate	$\eta_b = 5 \times 10^{-4}$
Hard-routing warmup ratio	$\rho_h = 0.2$
GRACE warmup ratio	$\rho_g = 0.01$
GRACE softmax beta	$\beta = 10$
GRACE blend exponent	$\lambda = 0.4$
Score-spread fallback threshold	$\epsilon = 0.001$
Alignment threshold	$\delta = -0.005$
Seed	42
Image resolution / visual token budget	Processor defaults; no manual override

4.4 Evaluation Metrics

We report benchmark-specific official metrics for each dataset:

- **TextVQA: VQA Accuracy.** For a predicted answer $\hat{a}$, the per-question score is

$$s_{\text{VQA}} = \min \left(\frac{\#\text{annotators that provided } \hat{a}}{3}, 1 \right),$$

and the final score is the mean over all questions [9].

- **DocVQA: ANLS** (Average Normalized Levenshtein Similarity). The final ANLS is the average score over all samples [10]:

$$\text{NLS}(a, \hat{a}) = 1 - \frac{d_{\text{Lev}}(a, \hat{a})}{\max(|a|, |\hat{a}|)}, \quad s_{\text{ANLS}} = \begin{cases} \text{NLS}(a, \hat{a}), & \text{NLS} \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

Group	Methods	Teacher setup	Purpose
Non-distillation references	Original, full finetuning	No teacher	Student-only reference points
Same-family baselines	Forward KL, Reverse KL, Jensen-Shannon	SmolVLM2-2.2B-Instruct	Standard single-teacher objectives within the SmolVLM family
Different-family single-teacher baselines	ULD	Qwen2-VL-2B-Instruct	Cross-family transfer without multi-teacher routing
Heterogeneous multi-teacher baselines	Uniform averaging, SHM-CKA, reinforced selection, GRACE	Mixed teacher set from multiple VLM families	Static averaging vs. hidden-state matching vs. adaptive teacher assignment

Table 4.3: Baseline groups used in the evaluation chapter. The grouping separates student-only references, same-family KD, cross-tokenizer single-teacher transfer and heterogeneous multi-teacher methods.

- **ChartQA: Relaxed Accuracy.** A prediction is correct if it matches exactly for text answers, or if the relative error is within 5% for numeric answers:

$$\frac{|\hat{y} - y|}{|y|} \leq 0.05.$$

The final score is the proportion of correct predictions [11].

4.5 Baselines and Compared Methods

For consistency with Chapter 5, we organize the compared methods into four groups: non-distillation references, same-family distillation baselines, different-family single-teacher baselines, and heterogeneous multi-teacher baselines.

Table 4.3 summarizes the role of each baseline family before the method-specific descriptions below.

- **Non-distillation references.** The original student and the full-finetuning baseline provide the student-only reference points for each benchmark.
- **Same-family baselines.** Forward Kullback-Leibler (KL), reverse KL and Jensen-Shannon divergence all use **SmolVLM2-2.2B-Instruct** as a same-family teacher [17, 22, 48, 59, 60]. They isolate standard teacher-student transfer when tokenizer, vocabulary and output space are already matched.
- **Different-family single-teacher baselines.** This group uses one teacher outside the SmolVLM family. ULD is the main cross-tokenizer logits baseline [26].

- **Heterogeneous multi-teacher baselines.** This group uses teacher sets drawn from different VLM families. Uniform averaging keeps fixed equal weights. SHM-CKA adds hidden-state matching over that set. Reinforced selection learns a stochastic teacher policy with REINFORCE [36, 61]. GRACE adds sparse routing, trie-Wasserstein KD and alignment-aware reweighting [27, 28, 37, 38].

For the trainable competitor methods, we keep the supervised term $\mathcal{L}_{CE}$. The logits-only baselines differ only in the KD signal, but SHM-CKA also adds a hidden-state alignment term and reinforced selection also adds a REINFORCE-style policy term [36, 40, 61]. In the descriptions below, γ denotes a generic KD mixing coefficient; α remains reserved for the repository training script in Table 4.2.

The compared methods are:

- **Original:** direct evaluation of the released student, with no additional optimization. This measures the native capability of the compact model before any task adaptation or distillation.
- **Full finetuning:** uses $\mathcal{L} = \mathcal{L}_{CE}$ with no teacher, so it isolates the benefit of external supervision alone.
- **Forward KL:** the standard same-family KD baseline, with $\mathcal{L}_{KD} = \text{KL}(p_t \| p_s)$ [22]. It tests soft-target transfer when tokenizer and vocabulary are already matched.
- **Reverse KL:** uses $\mathcal{L}_{KD} = \text{KL}(p_s \| p_t)$ [60, 62]. This variant is more mode-seeking and serves as a deliberately harder divergence baseline.
- **Jensen-Shannon:** uses the symmetric divergence $\frac{1}{2}\text{KL}(p_s \| m) + \frac{1}{2}\text{KL}(p_t \| m)$, where $m = \frac{1}{2}(p_s + p_t)$ [48, 60]. It tests whether a bounded symmetric objective is more stable than KL.
- **ULD:** uses $\mathcal{L}_{KD} = \|\pi(p_s) - \pi(p_t)\|_1$ [26]. This is the main cross-tokenizer logits baseline because it compares ranked probability mass, not token IDs.
- **SHM-CKA:** keeps the heterogeneous teacher set used by GRACE, but replaces routed weighting with a per-teacher ULD logits loss plus a hidden-state CKA penalty [26, 28, 40]. It tests whether direct representation alignment is sufficient without routed GRACE weighting.
- **Uniform multi-teacher:** averages the heterogeneous teacher losses with fixed equal weights, $\mathcal{L}_{KD} = \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{KD}^{(i)}$ [36, 37]. It isolates the effect of teacher diversity without adaptive assignment.

- **Reinforced selection:** replaces deterministic routing with a stochastic policy that samples teachers and updates the selector with REINFORCE [36, 61]. This baseline is close in spirit to adaptive selection, but it does not use GRACE score refinement.

For SHM-CKA, the historical training code first loads an offline CKA matrix and, for each student layer s , selects the top- k teacher layers $\mathcal{T}_s$. Their CKA scores are normalized into soft weights $\alpha_{s,t}$. The per-teacher layer loss is

$$\mathcal{L}_{\text{layer}}^{(i)} = \frac{1}{|S|} \sum_{s \in S} \sum_{t \in \mathcal{T}_s} \alpha_{s,t} (1 - \sqrt{\text{CKA}(\mathbf{H}_s, \mathbf{H}_t)}). \quad (4.1)$$

Each teacher also contributes its own ULD logits loss $\mathcal{L}_{\text{ULD}}^{(i)}$. In the historical single-teacher code, the default fixed-weight objective gives equal weight to CE, logits KD and layer matching:

$$\mathcal{L}_{\text{fixed}} = \frac{1}{3} (\mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{ULD}} + \mathcal{L}_{\text{layer}}). \quad (4.2)$$

That code also supports an alternative adaptive weighting scheme, but the fixed equal-mean form above is the default historical setting. The original soft top- k matching path was implemented for single-teacher runs, so the heterogeneous baseline used here keeps the hidden-state idea per teacher, averages the logits and layer terms across the teacher set,

$$\bar{\mathcal{L}}_{\text{ULD}} = \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{\text{ULD}}^{(i)}, \quad \bar{\mathcal{L}}_{\text{layer}} = \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{\text{layer}}^{(i)}, \quad (4.3)$$

then applies an equal-mean combination

$$\mathcal{L}_{\text{SHM-CKA}} = \frac{1}{3} (\mathcal{L}_{\text{CE}} + \bar{\mathcal{L}}_{\text{ULD}} + \bar{\mathcal{L}}_{\text{layer}}). \quad (4.4)$$

For reinforced selection, the policy uses the pooled student hidden state as a sample descriptor, then concatenates per-teacher summary features:

$$\mathbf{z} = [\text{Pool}(\mathbf{H}_s); \mathbf{f}_1; \dots; \mathbf{f}_M], \quad \mathbf{f}_i = [\mathcal{L}_{\text{KD}}^{(i)}, c_i, e_i], \quad (4.5)$$

where c_i is the teacher’s mean token confidence and e_i is its normalized token entropy over supervised positions. A linear policy then predicts independent Bernoulli probabilities

$$\boldsymbol{\pi} = \sigma(W \text{LN}(\mathbf{z}) + b). \quad (4.6)$$

During the warmup stage, all teachers stay active. After warmup, the baseline samples $m_i \sim \text{Bernoulli}(\pi_i)$, falls back to the highest-probability teacher if $\sum_i m_i = 0$ and normalizes the selected mask into weights

$$w_i = \frac{m_i}{\sum_{j=1}^M m_j}. \quad (4.7)$$

The multi-teacher KD term is the weighted average

$$\mathcal{L}_{KD} = \sum_{i=1}^M w_i \mathcal{L}_{KD}^{(i)}. \quad (4.8)$$

In the active launch script, $\mathcal{L}_{KD}^{(i)}$ is the per-teacher trie-Wasserstein loss. The selector is trained with a REINFORCE-style term

$$\mathcal{L}_{\text{pol}} = -(r - b) \log p(\mathbf{m} \mid \boldsymbol{\pi}), \quad (4.9)$$

where b is an EMA reward baseline. The active script uses $r = -\mathcal{L}_{CE} - \frac{1}{\sum_i m_i} \sum_i m_i \mathcal{L}_{KD}^{(i)}$, so the total reinforced-selection objective is

$$\mathcal{L}_{\text{RS}} = \mathcal{L}_{CE} + \gamma \mathcal{L}_{KD} + \lambda_{\pi} \mathcal{L}_{\text{pol}}. \quad (4.10)$$

Unless noted otherwise, all baselines reuse the data conversion pipeline, hardware, optimizer family and checkpointing protocol described earlier in this chapter. Methods that do not use routing simply omit the GRACE-specific terms and retain the corresponding single-teacher or uniform multi-teacher objective from the cited source.

4.6 Implementation Details

This section records the engineering details needed to reproduce the active launch path. These details are useful for systems reproducibility, but they are separate from the fair-comparison protocol summarized earlier.

4.6.1 Offline Teacher-Logit Preparation

Teacher logits are generated *offline* before distillation. An offline caching script runs each frozen teacher on every training example, keeps only the supervised answer-token positions and writes one `.pt` file per sample under a teacher-specific directory. Each record stores the sample index, the selected teacher logits and the corresponding supervised labels. This keeps the online training loop simple: the student only reads precomputed teacher targets instead of running the teachers again.

After generation, the teacher-logit directories are uploaded to a **Cloudflare R2** bucket using its S3-compatible object-storage interface [63, 64]. In the active setup, the bucket already exceeds **1 TB**, so copying the full cache to a rented GPU instance is impractical. This is especially relevant on services such as Vast.ai, where local NVMe space is usually much smaller than the full teacher logit store.

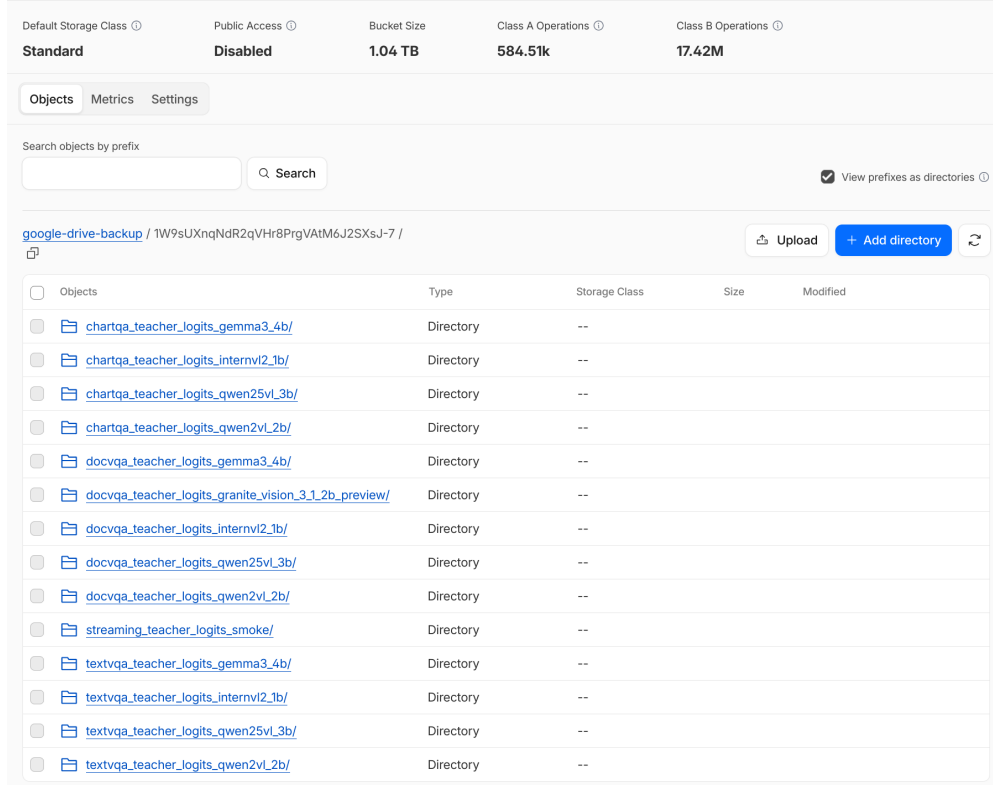


Figure 4.4: Cloudflare R2 bucket used for offline teacher-logit storage.

The remote bucket layout used for the offline teacher caches appears in Figure 4.4.

4.6.2 Remote Logit Streaming During Training

The main training path does *not* predownload the full cache. When the launcher receives an R2 root in `distill_multi_teachers.sh`, the trainer switches to remote lazy loading. During training, `StreamingTeacherLogitsCache` resolves the teacher-specific remote object path

`remote_root/<dataset>_teacher_logits_<alias>/<teacher-slug>/<index>.pt,`

then fetches files lazily from R2 through `s3fs` and `WholeFileCacheFileSystem` [65, 66]. The repository also includes a repacking utility for MDS, but the active training path reads the raw per-sample `.pt` cache directly.

To keep disk usage bounded, the loader uses a **100 GB** local cache. The loader caches each file only after first access, updates its access time after opening and removes the oldest files first once the cache exceeds the limit. In effect, the local cache behaves as a small least-recently-used style buffer over the much larger R2 store. A per-teacher file lock also guards cache metadata updates, so concurrent dataloader workers do not corrupt the cache state.

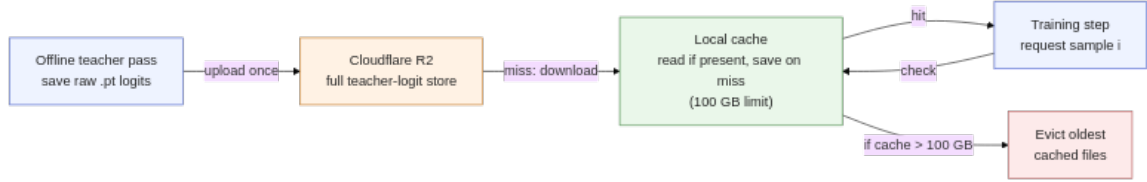


Figure 4.5: Training with remote teacher logits.

The remote-cache training path appears in Figure 4.5.

Table 4.4: Engineering settings for the active three-teacher DocVQA logit-streaming pipeline. These values describe storage, caching and launch-path behavior rather than the fair-comparison protocol itself.

Setting	Value
Teacher-logit storage	Cloudflare R2 object storage
Teacher-logit format	Raw per-sample .pt files
Remote path template	<code>remote_root/<dataset>_teacher_logits_<alias>/<teacher-slug>/<index>.pt</code>
Remote filesystem stack	<code>s3fs + WholeFileCacheFilesystem</code>
Optional repacking utility	MDS repacking is available but not used in the active path
Local cache directory	<code>/tmp/teacher-logits-streaming</code> for R2 reads <code>/workspace/cache</code> for local-cache reads
Local cache limit	100 GB
Cache eviction rule	Remove the oldest cached files first
Cache synchronization	Per-teacher file lock on cache metadata updates

Chapter 5

Experiment Results

This chapter reports the empirical results and analyzes what drives them. It first compares GRACE with the baseline methods on the main benchmarks, then examines stability, teacher usage and ablations. It closes with a discussion of the main gains, limitations and remaining gaps.

5.1 Main Comparison Results

This section answers RQ1 from Chapter 1 by comparing GRACE with same-family logits KD, cross-tokenizer ULD and the multi-teacher baselines on TextVQA, DocVQA and ChartQA.

5.1.1 Baseline Results

Before applying adaptation or distillation, we report the original performance of the two student models, **SmolVLM-500M** and **SmolVLM-256M**.

These baseline results establish the starting point for later comparison.

We obtain all results with **VLM-Eval-Kit** [53] under the evaluation pipeline described in Chapter 4.

Table 5.1 gives the student-only starting point.

The teacher set is substantially stronger than the compact students, which is what makes distillation meaningful in the first place.

Table 5.2 provides the corresponding teacher-side reference.

5.1.2 Overall Comparison

Tables 5.3, 5.4 and 5.5 summarize the main comparison results across original models, direct fine-tuning, standard logits distillation baselines and GRACE.

Model	TextVQA	DocVQA	ChartQA
SmolVLM-500M	56.75	59.52	62.80
SmolVLM-256M	47.17	48.64	45.04

Table 5.1: Original benchmark results of the two SmolVLM students on TextVQA, DocVQA and ChartQA. These scores are the student-only reference point for all later fine-tuning and distillation comparisons.

Teacher Model	TextVQA	DocVQA	ChartQA
Qwen2.5-VL-3B-Instruct	79.30	93.90	94.24
InternVL3-1B	74.10	81.90	75.30
Gemma-3-4B-it	57.80	75.80	63.60
Qwen2-VL-2B-Instruct	79.70	90.10	89.28
Granite Vision 3.1 2B	88.00	76.00	86.00

Table 5.2: Original benchmark results of the teacher and large reference models used here. The table establishes the teacher-side reference range used later in the gap-to-teacher analysis.

Unless stated otherwise, the main comparison tables report the strongest completed run for each method and benchmark. SHM-CKA, reinforced selection and GRACE use a four-teacher mixed set spanning the Qwen, InternVL and Gemma families. The Uniform baseline is reported from a cheaper two-teacher Qwen run. Chapter 4 documents the active three-teacher DocVQA launch script for systems and reproducibility details.

We report the baselines in four groups.

- The **non-distillation references** are the original student and the full-finetuning baseline used for comparison.
- The **same-family baselines** use **SmolVLM2-2.2B-Instruct** as teacher for forward KL, reverse KL and Jensen-Shannon divergence [22, 23, 28].
- The **different-family single-teacher baselines** use a teacher outside the SmolVLM family. This group includes the ULD baseline [26].
- The **multi-teacher baselines** compare a cheaper two-teacher Uniform average with four-teacher adaptive methods such as SHM-CKA, reinforced selection and GRACE [36–38, 61].

We report each configuration as a single run. Multi-teacher training is expensive in both GPU time and teacher-logit storage, so standard deviations are not available for every compared method.

Because each multi-teacher run requires large offline logit caches and long single-GPU training, we report one fully trained run per method. A typical H200 run takes about 7 hours, which is about 21 USD on Vast.ai before storage and offline-logit generation cost. We therefore treat very small differences cautiously, focus on consistent multi-point gaps across datasets and student sizes and leave broader sweeps for later work.

On DocVQA, GRACE rises from 69.59 to 73.11 on SmolVLM-500M and from 55.70 to 60.48 on SmolVLM-256M relative to the Uniform baseline.

ChartQA shows the largest margins over Uniform: 75.60 to 80.20 for SmolVLM-500M and 63.68 to 72.32 for SmolVLM-256M.

The tables above report end-point accuracy. Figure 5.4 adds one illustrative optimization trace.

Several trends are consistent across the three benchmarks:

- Same-family logits baselines remain weak. Forward KL gives only small gains, reverse KL is consistently poor and Jensen-Shannon is unstable. Simply matching a same-family teacher distribution is not enough for these compact students.
- **Cross-family ULD is already stronger than the same-family logits losses on all three tasks.** This shows that cross-tokenizer distillation is more useful here than vocabulary-matched KL. Once multiple teachers are added, Uniform averaging gives another clear jump, which means teacher diversity itself is already valuable.
- **The remaining jump from the single-teacher ULD baseline to GRACE comes from the full recipe: a trie-Wasserstein KD loss, sample-dependent sparse routing and GRACE score refinement after routing.**
- **GRACE consistently improves on the uniform multi-teacher baseline for both student sizes.** The gain is moderate on TextVQA and DocVQA, but larger on ChartQA. This suggests that adaptive weighting helps most when the task mixes visual reading, numerical reasoning and teacher specialization.

Category	Method	Teacher	Student	TextVQA			
Non-distillation references							
Original	None	None	SmolVLM-500M	56.75			
			SmolVLM-256M	47.17			
Reference	Finetuning	None	SmolVLM-500M	56.96			
			SmolVLM-256M	49.83			
Same-family distillation baselines							
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	59.00			
			SmolVLM-256M	50.88			
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	44.73			
			SmolVLM-256M	29.75			
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	42.33			
			SmolVLM-256M	33.84			
Different-family single-teacher baselines							
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	61.49			
			SmolVLM-256M	53.95			
Multi-teacher baselines							
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	63.88			
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	54.57			
	SHM-CKA	Qwen2-VL-2B-Instruct	SmolVLM-500M	60.47			
		Qwen2.5-VL-3B-Instruct					
		InternVL3-1B			SmolVLM-256M	55.93	
		Gemma-3-4B-it					
	Reinforced selection	Qwen2-VL-2B-Instruct	SmolVLM-500M	63.26			
		Qwen2.5-VL-3B-Instruct					
		InternVL3-1B			SmolVLM-256M	56.31	
	GRACE		Gemma-3-4B-it	SmolVLM-500M	69.38		
			Qwen2-VL-2B-Instruct				
			Qwen2.5-VL-3B-Instruct			SmolVLM-256M	60.73
			InternVL3-1B				

Table 5.3: TextVQA results for the original students, fine-tuning, single-teacher baselines and multi-teacher methods. The table includes one two-teacher Uniform row and four-teacher adaptive rows.

Category	Method	Teacher	Student	DocVQA		
Non-distillation references						
Original	None	None	SmolVLM-500M	59.52		
			SmolVLM-256M	48.64		
Reference	Finetuning	None	SmolVLM-500M	61.96		
			SmolVLM-256M	42.03		
Same-family distillation baselines						
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	63.55		
			SmolVLM-256M	49.46		
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	29.28		
			SmolVLM-256M	23.28		
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	36.43		
			SmolVLM-256M	33.12		
Different-family single-teacher baselines						
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	65.28		
			SmolVLM-256M	52.24		
Multi-teacher baselines						
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	69.59		
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	55.70		
	SHM-CKA	Qwen2-VL-2B-Instruct	SmolVLM-500M	63.32		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	57.24
		Gemma-3-4B-it				
	Reinforced selection	Qwen2-VL-2B-Instruct	SmolVLM-500M	64.00		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	58.53
		Gemma-3-4B-it				
	GRACE	Qwen2-VL-2B-Instruct	SmolVLM-500M	73.11		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	60.48
		Gemma-3-4B-it				

Table 5.4: DocVQA results for the original students, fine-tuning, single-teacher baselines and multi-teacher methods. The rows follow the grouping in Table 5.3, including one two-teacher Uniform row and four-teacher adaptive rows.

Category	Method	Teacher	Student	ChartQA		
Non-distillation references						
Original	None	None	SmolVLM-500M	62.80		
			SmolVLM-256M	45.04		
Reference	Finetuning	None	SmolVLM-500M	63.00		
			SmolVLM-256M	49.68		
Same-family distillation baselines						
Same-family	Forward KL	SmolVLM2-2.2B	SmolVLM-500M	64.52		
			SmolVLM-256M	56.80		
	Reverse KL	SmolVLM2-2.2B	SmolVLM-500M	30.16		
			SmolVLM-256M	17.20		
	Jensen-Shannon	SmolVLM2-2.2B	SmolVLM-500M	43.75		
			SmolVLM-256M	50.93		
Different-family single-teacher baselines						
Cross-family	ULD	Qwen2-VL-2B-Instruct	SmolVLM-500M	66.92		
			SmolVLM-256M	60.96		
Multi-teacher baselines						
Multi-teacher	Uniform	Qwen2-VL-2B-Instruct	SmolVLM-500M	75.60		
		Qwen2.5-VL-3B-Instruct	SmolVLM-256M	63.68		
	SHM-CKA	Qwen2-VL-2B-Instruct	SmolVLM-500M	76.88		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	62.40
		Gemma-3-4B-it				
	Reinforced selection	Qwen2-VL-2B-Instruct	SmolVLM-500M	70.27		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	68.74
		Gemma-3-4B-it				
	GRACE	Qwen2-VL-2B-Instruct	SmolVLM-500M	80.20		
		Qwen2.5-VL-3B-Instruct				
		InternVL3-1B			SmolVLM-256M	72.32
		Gemma-3-4B-it				

Table 5.5: ChartQA results for the original students, fine-tuning, single-teacher baselines and multi-teacher methods. This table follows the method layout used in the TextVQA and DocVQA comparisons, including one two-teacher Uniform row and four-teacher adaptive rows.

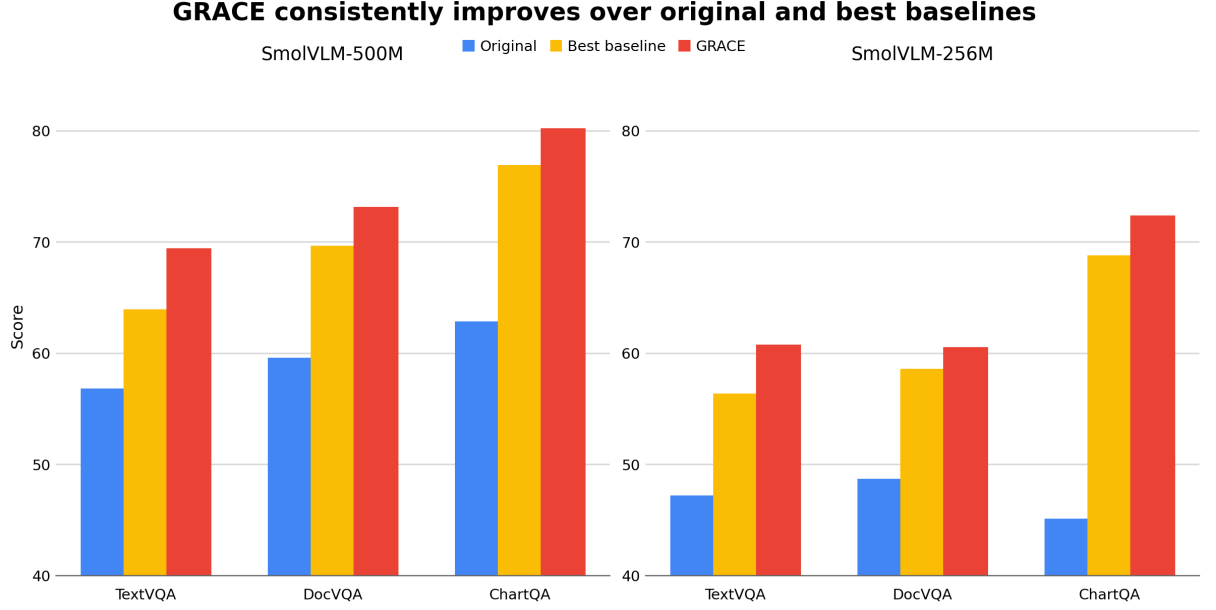


Figure 5.1: Grouped comparison of the original student, the strongest non-GRACE baseline and GRACE for both student sizes.

The bar chart in Figure 5.1 compresses the main ranking: GRACE stays above the strongest non-GRACE baseline on all three benchmarks and for both student sizes.

For the qualitative analysis below, we use DEX-AR [67]. For token t , layer l , and head h , DEX-AR forms a gradient-based attribution map

$$\mathbf{G}_t^{(l,h)} = \nabla_{\mathbf{A}^{(l,h)}} y_t \odot \mathbf{A}^{(l,h)},$$

then keeps visually grounded heads with

$$w^{(l,h)} = \max(S_{\text{img}}^{(l,h)} - S_{\text{text}}^{(l,h)}, 0),$$

and aggregates

$$\mathbf{H}_t = \sum_{l,h} w^{(l,h)} \mathbf{G}_t^{(l,h)}, \quad \mathbf{H}_{\text{sent}} = \sum_t \delta_t \mathbf{H}_t.$$

We prefer DEX-AR over raw attention maps or generic saliency because the model is autoregressive and the target is answer-token grounding, not generic image saliency.

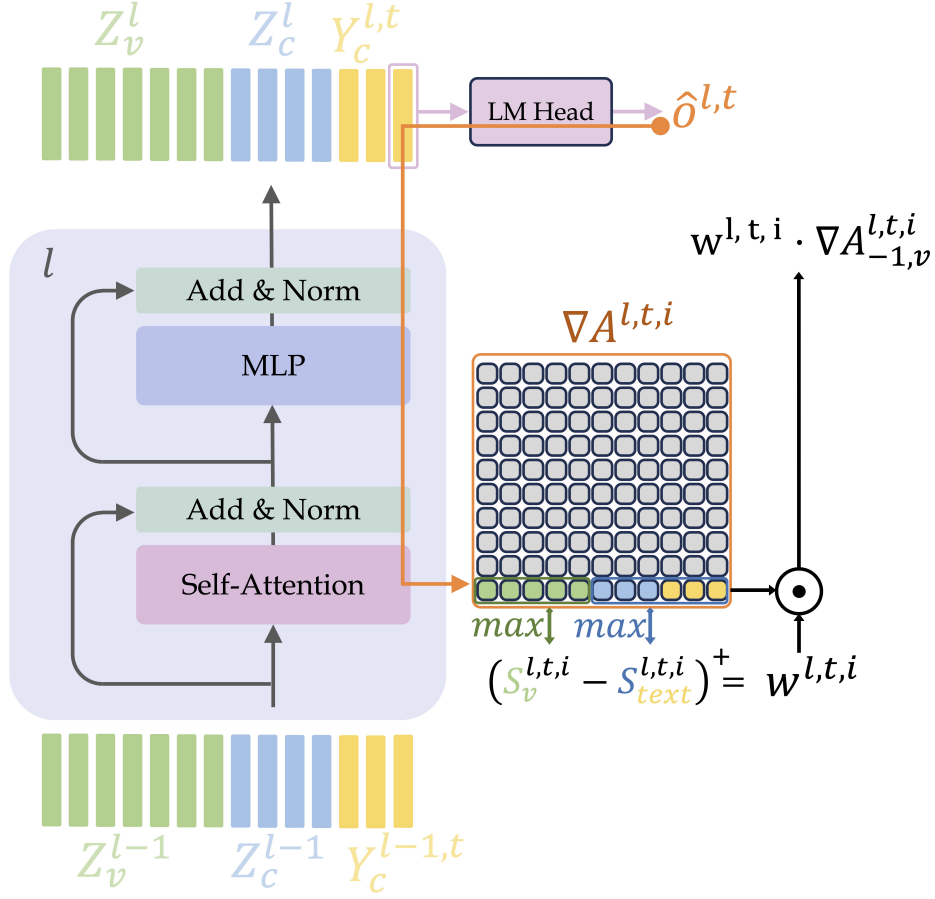


Figure 5.2: DEX-AR overview. The method computes gradient-based attribution over autoregressive attention maps, filters visually grounded heads and aggregates token-level maps into a sentence-level explanation.

Figure 5.3 gives a representative TextVQA example. Sample 3 asks “*who is this copyrighted by?*” The relevant evidence is the copyright text on the lower-left border. The original SmolVLM-500M instead attends to the *Virgin* logo, i.e. the wrong region. The GRACE model reduces that error.



(a) Input image



(b) Original SmolVLM-500M



(c) GRACE SmolVLM-500M

Figure 5.3: Representative TextVQA example for sample 3 (“*who is this copyrighted by?*”). The original SmolVLM-500M places highest mass on the salient *Virgin* logo, while the GRACE model shifts attention toward the copyright-text region.

Answer to RQ1

GRACE consistently improves over direct finetuning and the compared distillation baselines. The ranking is clear: same-family logits baselines are weakest, cross-family ULD is stronger, multi-teacher baselines improve further and GRACE is best overall. Within the multi-teacher group, static Uniform averaging already helps, but GRACE stays ahead of SHM-CKA, reinforced selection and Uniform itself. This shows that teacher diversity already helps, but the largest gains come from combining the trie-Wasserstein loss with gradient-based alignment. On SmolVLM-500M, GRACE closes 39.53% to 55.34% of the gap between the best teacher and the compact student.

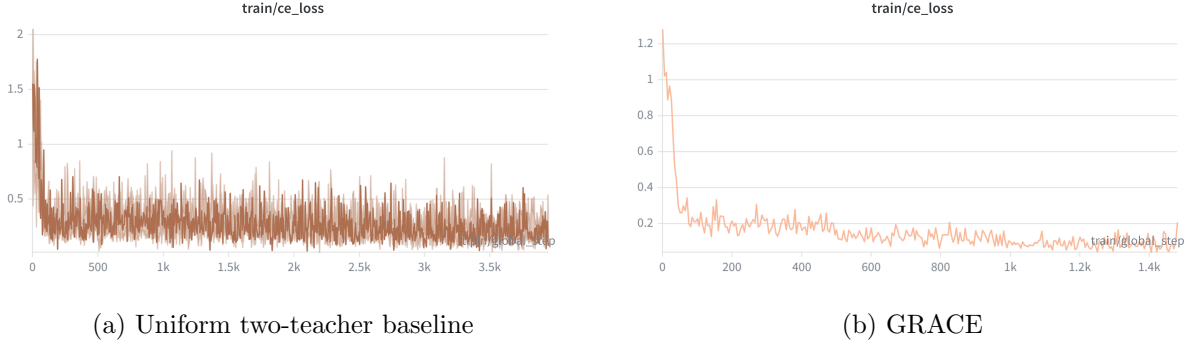
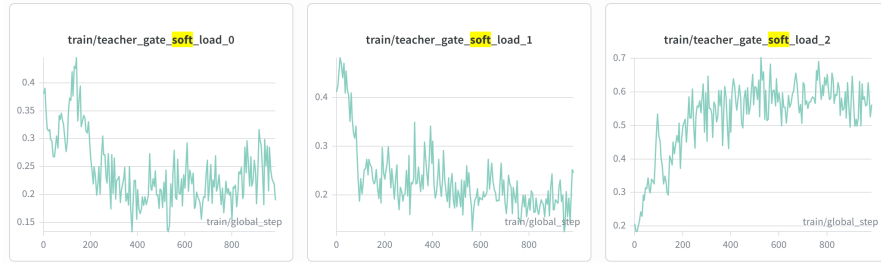


Figure 5.4: Illustrative training curves from a Uniform two-teacher run and one representative GRACE run.

The training-curve comparison in Figure 5.4 should be read as an example, not a matched stability study. In this pair of runs, the GRACE trace is smoother than the Uniform trace.



(a) Without GRACE refinement



(b) With GRACE

Figure 5.5: Representative routed-teacher load plots from one collapse case and one stable GRACE case.

The contrast is explicit in Figure 5.5. The figure is best read as a diagnostic contrast. In one routed run without the full GRACE recipe, the router drifts toward one teacher. In a representative GRACE run, the final load remains spread across teachers, so the student keeps the benefit of the heterogeneous pool.

Table 5.6 states the result numerically for the SmolVLM-500M student. Relative gain is computed as $(\text{GRACE} - \text{baseline})/\text{baseline}$.

- **The gains over the original student are large:** +22.26% on TextVQA, +22.83% on DocVQA and +27.71% on ChartQA. The gains over direct finetuning are almost as large: +21.80%, +18.00% and +27.30%, respectively.
- The gain over static uniform averaging is smaller, but it remains consistent across all three tasks. **This isolates the value of adaptive teacher weighting from the value of simply using more than one teacher.**

Benchmark	vs Original	vs Finetuning	vs Uniform	vs Best non-GRACE
TextVQA	+12.63 (+22.26%)	+12.42 (+21.80%)	+5.50 (+8.61%)	+5.38 (+8.41%)
DocVQA	+13.59 (+22.83%)	+11.15 (+18.00%)	+3.52 (+5.06%)	+3.52 (+5.06%)
ChartQA	+17.40 (+27.71%)	+17.20 (+27.30%)	+4.60 (+6.08%)	+3.32 (+4.32%)

Table 5.6: Absolute and relative gains of GRACE on SmolVLM-500M against the original model, direct finetuning, uniform weighting and the strongest non-GRACE baseline. Relative gain is computed as $(\text{GRACE} - \text{baseline})/\text{baseline}$.

Benchmark	Best teacher	Original gap	GRACE gap	Gap closed
TextVQA	79.70	22.95	10.32	55.03%
DocVQA	93.90	34.38	20.79	39.53%
ChartQA	94.24	31.44	14.04	55.34%

Table 5.7: Gap-to-teacher analysis for SmolVLM-500M under GRACE. The teacher reference is the strongest model in the four-teacher set for each benchmark and the last column reports the fraction of the original gap that GRACE closes.

- **The pattern also holds for SmolVLM-256M.** GRACE rises from 47.17 to 60.73 on TextVQA (+28.75%), from 48.64 to 60.48 on DocVQA (+24.34%) and from 45.04 to 72.32 on ChartQA (+60.57%).
- It also stays ahead of direct finetuning and uniform weighting. Relative to the uniform baseline, the gains are +11.29% on TextVQA, +8.58% on DocVQA and +13.57% on ChartQA.

The trend is similar if the reference point is the strongest teacher in the GRACE pool. Let $g_{\text{orig}} = y_T - y_{\text{orig}}$ and $g_{\text{grace}} = y_T - y_{\text{grace}}$. The fraction of the gap closed is $1 - g_{\text{grace}}/g_{\text{orig}}$.

The gain is not just a small increment over a weak baseline. On two benchmarks, GRACE removes about half of the original teacher-student gap. Even on DocVQA, where the teacher ceiling is highest, it still closes almost 40% of the gap.

The student can even outperform some individual teachers in the pool. On TextVQA, GRACE reaches 69.38 and exceeds Gemma-3-4B-it at 57.80 by +11.58 points (+20.03%). On ChartQA, it exceeds InternVL3-1B at 75.30 by +4.90 points (+6.51%) and Gemma-3-4B-it at 63.60 by +16.60 points (+26.10%). This shows that the final student is not simply bounded by the weaker teachers it learns from.

One negative result is also consistent: reverse KL is much weaker than forward KL. This suggests that the more mode-seeking objective is a poor fit for these compact multimodal students.

SHM-CKA is not enough by itself. SHM-CKA uses the heterogeneous teacher set, but replaces routed weighting with hidden-state matching. It is competitive on ChartQA, but clearly weaker on TextVQA and DocVQA. This implies that representation alignment alone is not sufficient. Teacher assignment matters, especially when the pool contains teachers with different strengths and tokenizers.

A clear limitation remains on the vision side. The method distills supervised answer-token logits, pools student hidden states for routing and scores teachers from CE/KD agreement at the output side. This is effective, but it does not explicitly align visual features, cross-attention patterns, or region-level evidence inside the VLM. The method therefore treats the vision encoder mostly through its effect on the final answer distribution, not as a first-class distillation target.

5.2 Representation Similarity Analysis

We use Centered Kernel Alignment (CKA) as an analysis tool to examine whether teacher and student layers exhibit clear correspondence. For centered hidden-state matrices $\tilde{\mathbf{H}}_s$ and $\tilde{\mathbf{H}}_t$, linear CKA is

$$\text{CKA}(\tilde{\mathbf{H}}_s, \tilde{\mathbf{H}}_t) = \frac{\left\| \tilde{\mathbf{H}}_t^\top \tilde{\mathbf{H}}_s \right\|_F^2}{\left\| \tilde{\mathbf{H}}_t^\top \tilde{\mathbf{H}}_t \right\|_F \left\| \tilde{\mathbf{H}}_s^\top \tilde{\mathbf{H}}_s \right\|_F}. \quad (5.1)$$

Values closer to 1 indicate stronger representation similarity [40].

The main question is whether strong internal similarity is enough to justify layer-wise distillation. Our results suggest that it is not, as shown in the figures and tables below.

The final-layer maps already show that the teacher and student are not arbitrarily far apart in representation space.

Teacher and student can remain relatively close in both the language and vision stacks, as Figures 5.6 and 5.7 indicate.

However, Tables 5.3, 5.4, and 5.5 still show a clear benchmark gap between compact students and stronger models. The layer-wise baseline also fails to consistently outperform output-level distillation.

This supports the design choice in GRACE. High hidden-state similarity alone is not a reliable signal that direct layer matching will produce better task transfer.

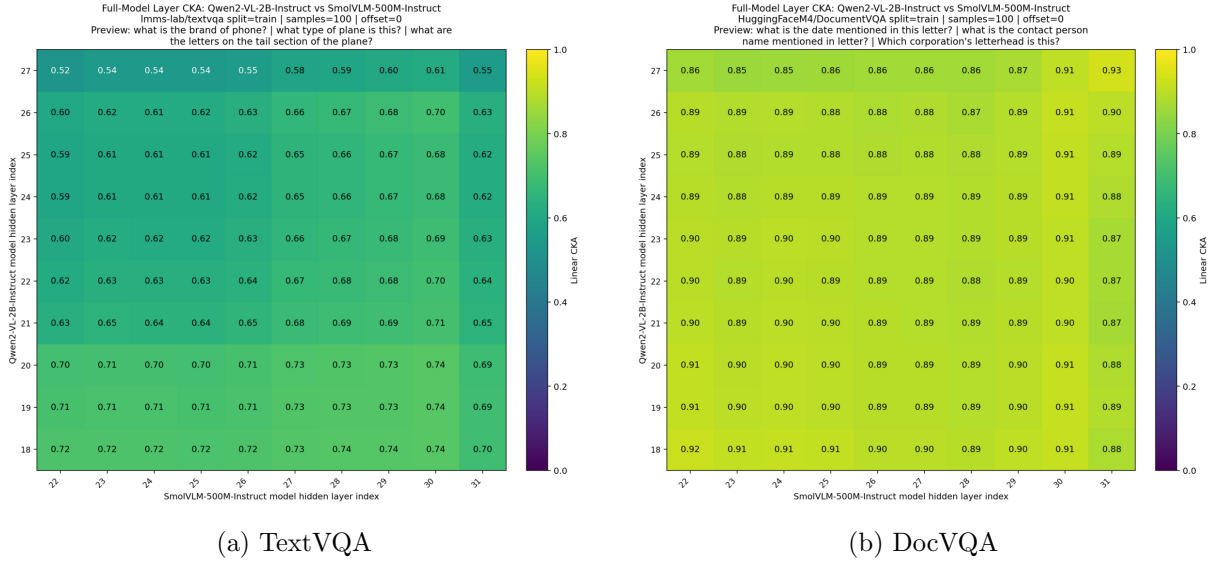


Figure 5.6: Final-layer CKA heatmaps for Qwen2-VL-2B and SmolVLM-500M on TextVQA and DocVQA.

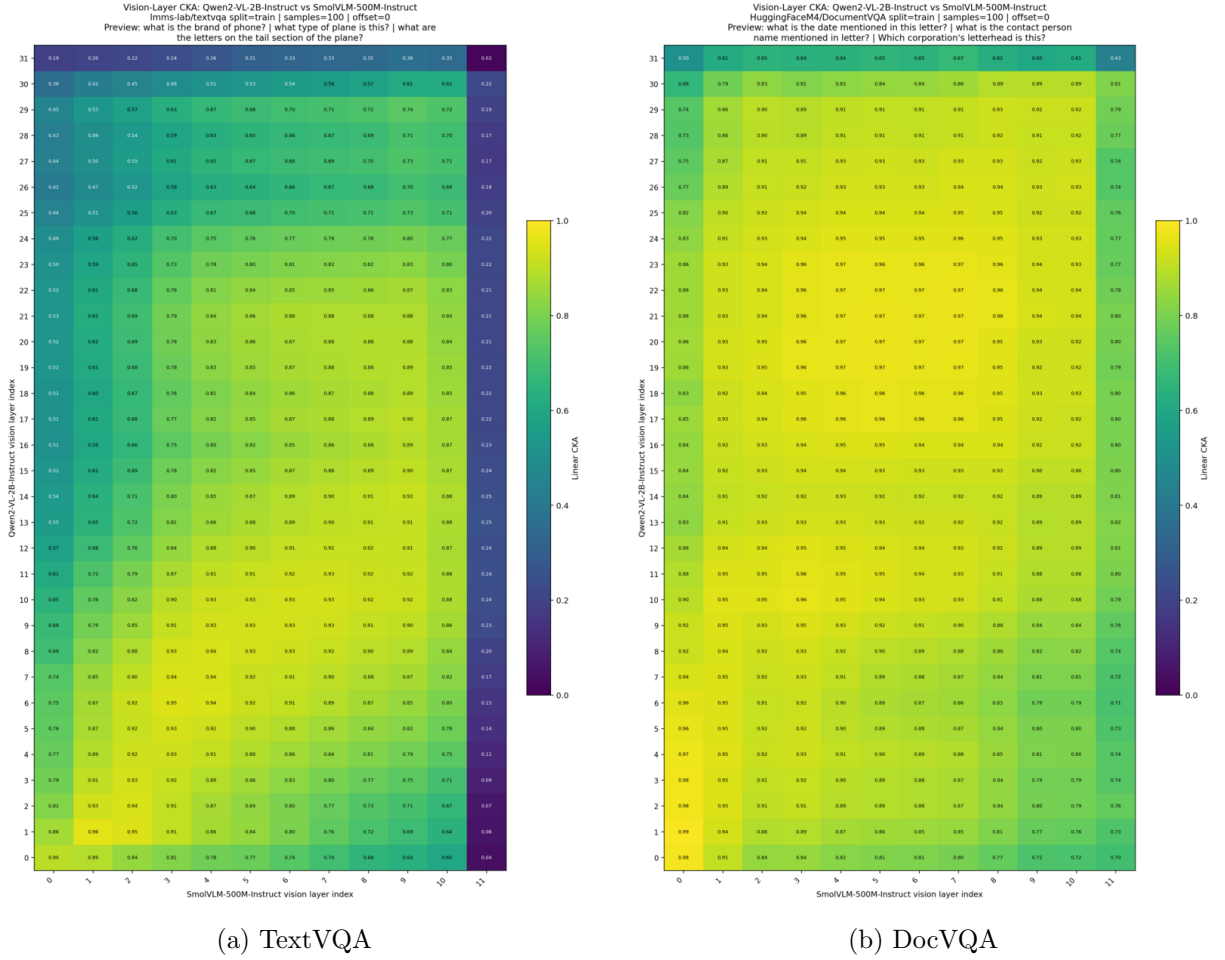


Figure 5.7: Vision-layer CKA heatmaps for Qwen2-VL-2B and SmolVLM-500M on TextVQA and DocVQA.

5.3 Ablation Study

This section answers RQ2 from Chapter 1 by isolating the main components of GRACE on DocVQA. Table 5.8 keeps the teacher set and KD loss close to the final method and changes one mechanism at a time:

- **Cross-family ULD (single teacher)** uses one cross-family teacher and optimizes $\mathcal{L} = \mathcal{L}_{CE} + \alpha \mathcal{L}_{ULD}$, with $\mathcal{L}_{ULD} = \|\pi(p_s) - \pi(p_t)\|_1$. This provides the single-teacher cross-tokenizer baseline.
- **Uniform multi-teacher** averages the active per-teacher KD losses, $\mathcal{L} = \mathcal{L}_{CE} + \alpha \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{TW}^{(i)}$. This tests whether teacher diversity alone explains the gain.
- **Routing only + trie-Wasserstein** keeps sparse routing but disables GRACE refinement after routing, so $w_i^{(b)} = u_i^{(b)} = \mathbf{1}[i \in \mathcal{R}_b]/|\mathcal{R}_b|$. This tests whether routing alone explains the gain.
- **Full GRACE + trie-Wasserstein** is the full method and serves as the reference row, with $w_i^{(b)} \propto (\hat{p}_i^{(b)})^\lambda (q_i^{(b)})^{1-\lambda}$.
- **Full GRACE + trie-Wasserstein** ($\lambda = 1.0$) reduces the final weights to router weights only, $w_i^{(b)} = \hat{p}_i^{(b)}$ after normalization over routed teachers. This tests whether post-routing blending matters.
- **Full GRACE + trie-Wasserstein** ($\lambda_{bal} = 0$) removes the router balance term, so $\mathcal{L} = \mathcal{L}_{CE} + \alpha \mathcal{L}_{KD} + \lambda_{ent} \mathcal{L}_{ent} + \lambda_z \mathcal{L}_z$. This tests whether anti-collapse regularization contributes beyond the main KD objective.
- **Full GRACE + trie-Wasserstein** ($\gamma_{EMA} = 0$) removes EMA smoothing, so $\hat{s}_i^{(b)} = s_i^{(b)}$ and $q_i^{(b)} = \text{Softmax}(\beta s_i^{(b)})$. This tests whether temporal score smoothing stabilizes teacher weighting.

Table 5.8 shows that the full variant remains the strongest setting for both student sizes. Relative to routing only, it gains 3.03 points on SmolVLM-256M and 9.88 points on SmolVLM-500M, which indicates that sparse routing alone does not explain the final improvement.

DocVQA ablation variant	SmolVLM-256M	SmolVLM-500M
Cross-family ULD (single teacher)	52.24	65.28
Uniform multi-teacher	55.70	69.59
Routing only + trie-Wasserstein	57.45	63.23
Full GRACE + trie-Wasserstein	60.48	73.11
Full GRACE + trie-Wasserstein ($\lambda = 1.0$)	57.97	70.45
Full GRACE + trie-Wasserstein ($\lambda_{bal} = 0$)	58.46	72.12
Full GRACE + trie-Wasserstein ($\gamma_{EMA} = 0$)	57.45	71.25

Table 5.8: DocVQA component ablation for SmolVLM-256M and SmolVLM-500M.

The table also reveals two distinct post-routing gains. On SmolVLM-500M, moving from routing-only weights to router-only weights ($\lambda = 1.0$) already adds 7.22 points ($63.23 \rightarrow 70.45$), and full GRACE adds another 2.66 points ($70.45 \rightarrow 73.11$). On SmolVLM-256M, the router-only gain is only 0.52 points ($57.45 \rightarrow 57.97$), while gradient-based refinement adds 2.51 points ($57.97 \rightarrow 60.48$). This suggests that the gate provides a coarse ranking, especially for the larger student, whereas gradient-based reweighting is the more universal corrective layer [36, 44]. This also suggests an interaction between student size and routing quality: the larger student appears to extract stronger routing features from its hidden states, while the smaller student relies more on the GRACE refiner. This is an inference from the ablation pattern, not a direct causal proof.

Removing router balance loss or EMA smoothing causes smaller but consistent degradations, suggesting that both terms mainly stabilize teacher weighting instead of replacing the main contribution of GRACE-guided blending.

Answer to RQ2

GRACE works because it combines a strong cross-tokenizer KD loss (trie-Wasserstein), sample-dependent sparse routing and, most critically, post-routing gradient-based score refinement. The balance loss and EMA smoothing are supportive stabilizers.

5.4 Efficiency Analysis

This section addresses RQ3 from Chapter 1. Runtime, training cost and storage overhead will be summarized here.

Answer to RQ3

The cost of GRACE is dominated by GPU training rather than object storage. From the Vast.ai screenshot, one H200 instance costs 2.059 USD/hour, so a 2500 USD budget corresponds to

$$\frac{2500}{2.059} \approx 1214.18 \text{ GPU-hours,}$$

which is about 50.59 days of continuous single-GPU runtime. If one full run takes about 7 hours, this budget is equivalent to roughly

$$\frac{1214.18}{7} \approx 173$$

single-H200 runs.

The storage side is much cheaper under the official Cloudflare R2 standard pricing policy: 0.015 USD/GB-month for storage, 4.50 USD per million Class A operations, 0.36 USD per million Class B operations and zero egress. Using the dashboard values in Figure 4.4, 1.04 TB of stored logits is about 1040 GB, which gives

$$1040 \times 0.015 = 15.60 \text{ USD/month.}$$

The 598.73k Class A operations stay below the 1M monthly free allowance, so their marginal cost is 0 USD. The 20.3M Class B operations exceed the 10M free allowance by 10.3M, which gives

$$10.3 \times 0.36 = 3.71 \text{ USD.}$$

The total R2 cost is therefore about

$$15.60 + 3.71 = 19.31 \text{ USD/month,}$$

before billing-unit rounding. This is below 1% of the 2500 USD GPU budget, which means the main efficiency bottleneck is training compute and offline logit generation, not storage egress or serving cost.

5.5 Discussion

- **The results show a clear ordering.** Same-family KL baselines are weakest, single cross-family distillation is stronger, uniform weighting improves again and GRACE is best overall [22, 26]. This means that the problem is not solved by one trick alone. Heterogeneous teachers provide better supervision, but the student still needs a cross-tokenizer loss, sample-dependent routing and a stable post-routing weighting scheme.
- The code-level implementation helps explain that pattern. The training recipe uses a shared-trie Wasserstein loss for mismatched tokenizers, a routed multi-teacher setup with hard-routing warmup, top- k capacity control and GRACE score refinement with EMA-smoothed score updates [4, 26, 42]. The smoother training curve in Figure 5.4 is consistent with that design: the router narrows the teacher set first, then GRACE reweights only the routed teachers instead of averaging them uniformly.
- The largest gain appears on ChartQA and the trend also holds for the smaller 256M student. This suggests that sample-dependent teacher assignment is most useful when the task mixes visual parsing and numerical reasoning. It also shows that the method is not only helping the larger compact student. The improvement persists in the more resource-constrained regime.
- The comparison with SHM-CKA is also informative. SHM-CKA uses the heterogeneous teacher set, but it is less consistent than GRACE. Together with the CKA heatmaps, this suggests that internal similarity alone is not enough. **Good transfer still depends on which teacher is emphasized for each sample** [28, 40].

5.6 Limitations and Threats to Validity

- **Cost-limited evaluation.** The reported numbers are single-run results. A typical H200 run takes about 7 hours and costs about 21 USD on Vast.ai before offline cache-generation and storage cost. This limits repeated seeds, confidence intervals and broader hyperparameter sweeps, so small gaps between strong baselines should be interpreted cautiously.

- **Incomplete apples-to-apples coverage across all baselines.** The strongest adaptive rows use the full heterogeneous teacher set, while the Uniform baseline is reported from a cheaper two-teacher run. The active Chapter 4 launch path also documents one three-teacher DocVQA recipe, whereas the main result tables aggregate broader completed runs. This does not invalidate the overall trend, but it weakens strict per-row comparability.
- **Answer-token-centric transfer.** The method distills teacher distributions on supervised answer positions, routes teachers from pooled student states and refines them with CE/KD gradient agreement. It does not explicitly align visual tokens, cross-attention maps or region-level evidence. As a result, some gains may come mainly from output-side transfer instead of deeper visual grounding.
- **Systems dependence.** The implementation relies on large offline teacher-logit caches, remote object storage and local streaming caches. This keeps the online loop simple, but it raises storage, I/O and reproducibility requirements. The method is therefore easier to reproduce in a well-provisioned training environment than on minimal local hardware.
- **Scope of validation.** The empirical study focuses on text-rich VQA benchmarks and compact students in the SmolVLM family. The reported gains are strong in this regime, but the thesis does not yet establish the same behavior for other multimodal task families, other student architectures or larger teacher pools.

Conclusion and Future Work

This chapter summarizes the main contributions and the empirical conclusions of the study. The main limitations are discussed separately in Chapter 5. This final part focuses on the core takeaways and on practical directions for future work.

We studied compact visual language model distillation under heterogeneous teachers. The student is much smaller than the teacher set, the teachers do not help every sample equally and their tokenizers do not match. We present GRACE, a routed multi-teacher framework with trie-Wasserstein cross-tokenizer distillation, capacity-aware selection and EMA-smoothed GRACE weighting. The code follows that order: route teachers first, refine their weights with GRACE scores, then aggregate the routed trie-Wasserstein KD loss [4, 26, 42].

Empirically, GRACE is the strongest method in the main comparison. For SmolVLM-500M, it reaches 69.38 on TextVQA, 73.11 on DocVQA and 80.20 on ChartQA. For SmolVLM-256M, it reaches 60.73, 60.48 and 72.32. These results improve over the original students, direct finetuning and the best competing baselines in Chapter 5. On SmolVLM-500M, GRACE closes about 55% of the original teacher-student gap on TextVQA and ChartQA and about 40% on DocVQA. Taken together, these results establish that selective routing plus post-routing score refinement is a strong recipe for compact text-rich VLM distillation.

Future Work

- Stronger evaluation. Repeated runs would make the comparison more rigorous, especially for the smaller gaps between the stronger multi-teacher baselines.
- Better systems efficiency. The pipeline still relies on large offline teacher-logit caches, so reducing storage and I/O cost would make the method easier to scale to more datasets and larger teacher sets.
- Broader method extensions. The recipe is still answer-token centric, so one important next step is explicit visual distillation: visual-token alignment, cross-attention transfer, or region-level matching. The method can also study richer router inputs, stronger objective-conflict handling and teacher sets that mix specialized document, chart and general VLM experts [36–38].

References

- [1] Wikimedia Commons contributors. File: Contrastive language-image pretraining.png. Wikimedia Commons, 2024. MIT License. Accessed: 2026-03-06.
- [2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.
- [3] Nicolas Papadakis. File: Continuous optimal transport.png. Wikimedia Commons, 2018. CC BY-SA 4.0. Source image from N. Papadakis, *Optimal Transport for Image Processing*, 2015. Accessed: 2026-04-15.
- [4] Gabriel Peyré and Marco Cuturi. Computational optimal transport, 2019.
- [5] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning, 2022.
- [6] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023.
- [7] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023.
- [8] Yushi Hu, Otilia Stretcu, Chun-Ta Lu, Krishnamurthy Viswanathan, Kenji Hata, Enming Luo, Ranjay Krishna, and Ariel Fuxman. Visual program distillation: Distilling tools and programmatic reasoning into vision-language models, 2024.
- [9] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read, 2019.
- [10] Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. Docvqa: A dataset for vqa on document images, 2021.

- [11] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning, 2022.
- [12] Xiangxiang Chu, Limeng Qiao, Xinyang Lin, Shuang Xu, Yang Yang, Yiming Hu, Fei Wei, Xinyu Zhang, Bo Zhang, Xiaolin Wei, and Chunhua Shen. Mobilevlm: A fast, strong and open vision language assistant for mobile devices, 2023.
- [13] Baichuan Zhou, Ying Hu, Xi Weng, Junlong Jia, Jie Luo, Xien Liu, Ji Wu, and Lei Huang. Tinyllava: A framework of small-scale large multimodal models, 2024.
- [14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.
- [15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022.
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008, 2017.
- [17] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Vaibhav Srivastav, Joshua Lochner, Hugo Larcher, Mathieu Morlon, Lewis Tunstall, Leandro von Werra, and Thomas Wolf. Smolvlm: Redefining small and efficient multimodal models, 2025.
- [18] Andrés Marafioti, Merve Noyan, Miquel Farré, Elie Bakouch, and Pedro Cuenca. Smolvlm - small yet mighty vision language model. Hugging Face Blog, 2024. Accessed: 2026-04-02.
- [19] AI Rank. Textvqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.
- [20] AI Rank. Docvqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.

- [21] AI Rank. Chartqa benchmark: Complete leaderboard and performance analysis. AI Rank, 2026. Self-reported leaderboard. Accessed: 2026-04-02.
- [22] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- [23] Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models, 2024.
- [24] Zi Wang. Zero-shot knowledge distillation from a decision-based black-box model. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10675–10685. PMLR, 2021.
- [25] Wenhao Ma, Jie Li, Bo Han, and Xiaojuan Qi. B2kd: A unified framework for knowledge distillation from black-box models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15313–15323, 2024.
- [26] Nicolas Boizard, Kevin El Haddad, Pierre Colombo, and Chloé Clavel. Towards cross-tokenizer distillation: The universal logit distillation loss for llms, 2024.
- [27] Yuki Takezawa, Ryoma Sato, and Makoto Yamada. Supervised tree-wasserstein distance. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10086–10095. PMLR, 2021.
- [28] Sayantan Dasgupta and Trevor Cohn. Improving language model distillation through hidden state matching. In *International Conference on Learning Representations*, 2025.
- [29] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2022.
- [30] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, pages 38087–38099, 2023.
- [31] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2024.

- [32] Shiyao Li, Yingchun Hu, Xuefei Ning, Xihui Liu, Ke Hong, Xiaotao Jia, Xiuhong Li, Yaqi Yan, Pei Ran, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. Mbq: Modality-balanced quantization for large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4167–4177, 2025.
- [33] Zhiyuan Fang, Jianfeng Wang, Xiaowei Hu, Lijuan Wang, Yezhou Yang, and Zicheng Liu. Compressing visual-linguistic model via knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1428–1438, 2021.
- [34] Kaicheng Yang, Tiancheng Gu, Xiang An, Haiqiang Jiang, Xiangzi Dai, Ziyong Feng, Weidong Cai, and Jiankang Deng. Clip-cid: Efficient clip distillation via cluster-instance discrimination, 2024.
- [35] Wikimedia Commons contributors. File: Knowledge-distillation-example.png. Wikimedia Commons, 2023. CC0 1.0. Source: Intel Distiller knowledge distillation page. Accessed: 2026-03-07.
- [36] Fei Yuan, Linjun Shou, Jian Pei, Wutao Lin, Ming Gong, Yan Fu, and Daxin Jiang. Reinforced multi-teacher selection for knowledge distillation, 2020.
- [37] Hailin Zhang, Defang Chen, and Can Wang. Adaptive multi-teacher knowledge distillation with meta-learning, 2023.
- [38] Yu-Chu Yu, Chi-Pin Huang, Jr-Jen Chen, Kai-Po Chang, Yung-Hsuan Lai, Fu-En Yang, and Yu-Chiang Frank Wang. Select and distill: Selective dual-teacher knowledge transfer for continual learning on vision-language models, 2024.
- [39] Tuan Le, Makoto Yamada, Kenji Fukumizu, and Marco Cuturi. Tree-sliced variants of wasserstein distances. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [40] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529. PMLR, 2019.
- [41] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

- [42] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [43] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing, 2022.
- [44] Yichen Zhu and Yi Wang. Student customized knowledge distillation: Bridging the gap between student and teacher. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5057–5066, 2021.
- [45] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [46] Wikimedia Commons contributors. File: Transformer, attention block diagram.png. Wikimedia Commons, 2024. CC BY 4.0. Accessed: 2026-03-06.
- [47] Wikimedia Commons contributors. File: Vision transformer.png. Wikimedia Commons, 2024. CC BY 4.0. Accessed: 2026-03-06.
- [48] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory*, 37(1), 1991.
- [49] Wikimedia Commons contributors. File: Optimal transport matrix.png. Wikimedia Commons, 2018. CC BY-SA 4.0. Accessed: 2026-03-31.
- [50] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [51] Wikimedia Commons contributors. File: Deepseek moe and mla (deepseek-v2).svg. Wikimedia Commons, 2025. MIT License. Accessed: 2026-03-06.
- [52] Zerodamage. File: Gradient descent.svg. Wikimedia Commons, 2012. Public domain. Accessed: 2026-03-31.
- [53] Haodong Duan, Xinyu Fang, Junming Yang, et al. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models, 2024.
- [54] Wikimedia Commons contributors. File: Stop sign.jpg. Wikimedia Commons, 2006. Public domain. Accessed: 2026-03-06.

- [55] Wikimedia Commons contributors. File: Receiptswiss.jpg. Wikimedia Commons, 2015. CC BY-SA 3.0. Accessed: 2026-03-06.
- [56] Wikimedia Commons contributors. File: Bar chart.svg. Wikimedia Commons, 2016. CC BY-SA 4.0. Accessed: 2026-03-06.
- [57] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020.
- [58] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016.
- [59] Hugging Face TB. Smolvlm2-2.2b-instruct model card. Hugging Face, 2025. Accessed: 2026-04-02.
- [60] Yifan Cui, Wei Wang, Yifei Wang, Yao Lu, Haoyu Wang, Fuzhen Zhuang, Qing He, and Chuan Shi. Sinkhorn distance minimization for knowledge distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11046–11060. Association for Computational Linguistics, 2024.
- [61] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- [62] Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f-divergence minimization for sequence-level knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10817–10834. Association for Computational Linguistics, 2023.
- [63] Cloudflare. R2 object storage. Cloudflare developer documentation, 2026. Accessed: 2026-04-09.
- [64] Cloudflare. Use the aws sdk for python (boto3) with r2. Cloudflare developer documentation, 2026. Accessed: 2026-04-09.
- [65] fsspec developers. Features. fsspec documentation, 2026. Accessed: 2026-04-09.
- [66] fsspec developers. fsspec.implementations.cached. fsspec documentation, 2026. Accessed: 2026-04-09.
- [67] Walid Bousselham, Rishikesh Jadhav, Simon de Mathelin, François Fleuret, and Pascal Frossard. Dex-ar: Explainability in multimodal large language models with deep expectation and attention rollout, 2026.